Universität Regensburg

Fakultät für Sprach-, Literatur- und Kulturwissenschaften

Bachelorarbeit

Im Studiengang Englische Sprachwissenschaft

Sommersemester 2026

Can AI Replace Traditional Textbooks?

A Comparative Corpus Analysis of LLM-Generated vs. Textbook Example Sentences

Autor:	Simon John
Matrikelnummer:	2175939
Adresse:	Wildbachweg 5, 93059 Regensburg
E-Mail:	simon.john@stud.uni-regensburg.de
Studiengang:	BA Englische Sprachwissenschaft
Abgabedatum:	May 5, 2026
Gutachter:	Dr. Thorsten Brato

Abstract

Can Large Language Models (LLMs) generate pedagogical input that is both register-appropriate and systematically different from traditional textbook materials? This is the question this study seeks to answer. The Natural Reference Corpus (NRC, $N = 5,483$) compiled from B1/B2 English as a foreign language (EFL) textbooks was contrasted with the Synthetic Control Corpus (SCC, $N = 1,237$), which was artificially generated by Google Gemini 2.5 Flash under the following register conditions: HIGH (formal workplace setting), NEUTRAL (instructional setting) and LOW (casual, conversational setting). For this study, the two corpora were filtered specifically for sentences containing ten selected high-frequency, highly polysemous verbs such as *run*, *take* and *make*. The verbs in these sentences were then classified as either LITERAL or IDIOMATIC via an LLM-as-a-judge protocol, which was subsequently validated through inter-rater reliability (IRR) testing ($\kappa = 0.62$). Statistical analysis confirmed three hypotheses. First, sentences generated by Artificial Intelligence (AI) yielded significantly more IDIOMATIC contexts than the textbook sentences ($\chi^2 = 143.90, p < 0.001$). Second, the NRC aligned most closely with the NEUTRAL subsection of the SCC ($V = 0.024$), hinting at a high use of a distinct instructional register in English textbooks. Third, rather than behaving uniformly, idiomaticity was lemma-selective in both corpora. The findings suggest that, if prompted explicitly, LLMs can approximate or even exceed textbook register behaviour. This "naturalness", however, depends heavily on communicative framing, rather than on model capability alone. Implications for future textbook design and AI-assisted pedagogy are discussed.

Table of Contents

List of Tables

List of Figures

List of Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CA	Communicative Approach
CEFR	Common European Framework of Reference for Languages
CLT	Cognitive Load Theory
CSV	Comma Separated Values
EFL	English as a Foreign Language
ETL	Extract, Transform, Load
GTM	Grammar-Translation Method
HIGH	High Register Condition
IDIOMATIC	Idiomatic Usage Label
IRR	Inter-Rater Reliability
JSON	JavaScript Object Notation
KWIC	Key Words in Context
LLM	Large Language Model
LITERAL	Literal Usage Label
LOW	Low Register Condition
MDA	Multi-Dimensional Analysis
NEUTRAL	Neutral Register Condition
NLP	Natural Language Processing
NLTK	Natural Language Toolkit
NONE	Invalid Context
NTP	Next-Token Prediction
NRC	Natural Reference Corpus
POS	Part-of-Speech
PPO	Proximal Policy Optimization
RLHF	Reinforcement Learning with Human Feedback
SCC	Synthetic Control Corpus

List of Abbreviations

Abbreviation	Meaning
TEC	Textbook English Corpus
QA	Quality Assurance
XML	eXtensible Markup Language

1 Introduction

1.1 Background & Motivation

Recently, the integration of Large Language Models (LLMs) into the language learning classroom is heavily reshaping how pedagogical content can be produced and distributed. In practical terms, this means that language learners are no longer limited to studying with example texts in textbooks, but can request context-specific alternatives on demand. While this development creates new opportunities for adaptive learning, it also reopens the discussion about a persistent question in applied linguistics: how closely do pedagogical examples resemble authentic language use in everyday communication?

This question is significant because traditional “Textbook English” has long been criticised for being overly stiff and unnatural (Conti and Smith 2016). Traditional learning materials tend to be explicit in meaning and syntactically strict, presenting a “saniti(s)ed” (Curdt-Christiansen and Weninger 2015: 13) version of English that rarely matches how speakers actually interact with each other. Here, Generative Artificial Intelligence (AI) offers a promising solution by generating “natural, casual” English when prompted accordingly. The question is, however, whether LLMs can truly recreate the phraseological nuances of authentic speech, or if they simply produce a statistically average imitation that is just as artificial and “sanitised” as the textbooks they aim to replace.

This tension between teachability and authenticity is not only about style. If learning materials overwhelmingly contain explicit grammatical and syntactical transparency over natural phraseology, learners receive input that is clear, but does not represent how language in different social contexts works. This is bound to cause intelligibility issues in real-world situations outside the English as a foreign language (EFL) classroom. Register-oriented (Biber 1988; Le Foll 2021b) and phraseological research (Sinclair 1991) therefore provide an appropriate basis for an empirical study on this issue.

1.2 Research Gap & Study Rationale

Most recent work regarding AI in education focuses on the technology as an evaluator, assistant or interactional partner for learner output (Holmes et al. 2019). In contrast, comparatively little research has been conducted on AI as a robust provider of pedagogical input. This is the central gap this thesis addresses.

The secondary gap is methodological. Claims that AI output either sounds "natural" or "unnatural" are often made intuitively without actual testing through objective corpus linguistics procedures. To move on from subjective intuition to quantitative evidence, this thesis operationalises authenticity through measurable linguistic dimensions, specifically focusing on register distribution (Biber 1988) and patterns in idiomaticity (Sinclair 1991).

From the perspective of language acquisition, the relevance of this gap is very practical. The input quality has a direct impact on what learners can notice and acquire over time (Krashen 1985). Even though this study does not measure cognitive load directly, it acknowledges that wording, phraseology and information packaging play a fundamental role in cognitive processing in educational contexts (Spiro et al. 1992; Sweller 1988).

1.3 Research Question & Design Overview

This study uses a data-driven approach, utilizing methods of Corpus Linguistics to address the following research question: *To what extent can Large Language Models (specifically Gemini 2.5 Flash) generate linguistically distinct registers (formal, instructional and casual) that quantitatively differ from human-authored textbook materials?* To answer this question, this study takes an existing Natural Reference Corpus (NRC), compiled by Le Foll (2021a) from EFL textbook materials and compares it to a Synthetic Control Corpus (SCC). The SCC is generated by Google's Gemini 2.5 Flash Application Programming Interface (API) via role-prompting across three distinct registers: formal workplace language (HIGH register: questions and professional contexts), instructional language (NEUTRAL register: imperatives, directives and rules) and casual conversational language (LOW register: statements in informal contexts). The comparative nature of this study enables testing for alignment between textbook English and the AI generated formal, instructional or casual registers. It might also reveal, if textbooks have a distinct pedagogical register unlike any of these.

In analytical terms, the comparison between the two corpora is made on sentence level, focusing on ten high-frequency, polysemous verbs (e.g., *run, take, make*). As such verbs are both common and semantically flexible (Stoeckel 2019: 54–55), research of their occurrences in literal versus idiomatic contexts will provide a quantifiable metric for phraseological authenticity. By tracking how these verbs shift between transparent, literal meanings and de-lexicalised, idiomatic chunks, the analysis will reveal whether pedagogical materials artificially suppress idiomaticity in favour of grammatical and lexical clarity, and whether the LLM replicates or diverges from this pattern across the generated registers.

1.4 Scope & Structure of the Thesis

The research question is deliberately limited to the phraseology of verbs at the intermediate learner level (B1/B2) for three methodological reasons. First, high-frequency, polysemous verbs have high variability in different contexts within controlled lexical sets (e.g., tracking a single lemma like *run* as it shifts between physical movement and de-lexicalised usage like *run out of time*). Second, intermediate-level material represents the turning point where formulaic competence and idiomaticity become relevant to develop a better understanding of a language (Schmale 2022: 108). Third, restricting the analysis to ten hand-picked lemmas enables intensive profiling of the different registers while still maintaining computational feasibility.

The remainder of the thesis is structured as follows. Section 2 establishes a necessary theoretical framework that will provide a valid anchor for the analysis and its interpretation. This section focuses on Biber's Register Theory, Sinclair's Idiom Principle and cognitive approaches to language input. Section 3 outlines the methodology of the analysis, including the compilation of the NRC and SCC. It also explains the Python-based ETL workflow and the LLM-as-a-judge evaluation system. Section 4 continues with the presentation of the results of the corpus comparison, which are then interpreted and evaluated against the study's hypotheses in Section 5. Finally, Section 6 summarises the findings, acknowledges methodological limitations and discusses implications for the design of future learning materials.

2 Theoretical Framework

This chapter sets a theoretical anchor used to evaluate the AI-generated example sentences against textbook input. Rather than portraying AI output as globally "good" or "bad", the framework makes the quality of the input measurable through three dimensions: register appropriateness, phraseological naturalness and cognitive manageability. These dimensions are used to make relevant claims about authenticity that can practically be tested by corpus analysis.

2.1 Register Theory: The Anatomy of "Casualness"

2.1.1 The Concept of Register

In linguistics, register refers to the variation of language in situational contexts: who is communicating, for what purpose and under which communicative constraints (Biber et al. 2021: 15). This is significantly different from the genre of a text, with which register is often confused. Genre typically refers to socially recognised types of text (e.g., newspaper article, novel, research paper) and their conventional structure. Fundamentally, genre tells us what kind of text we are looking at, while register tells us how language is produced under those specific circumstances.

This difference is important for the present thesis. If AI-generated content is evaluated without regard to the context and the prompt with which the output has been generated, judgements on its "naturalness" are not valid. A sentence that is totally normal in a formal instructional setting might be inappropriate in a casual conversation, and vice versa. Therefore, this analysis is based on a register-relative approach rather than making absolute claims.

Textbook English has a distinct instructional register that systematically differs from non-pedagogical registers (Le Foll 2021b). Le Foll's studies in EFL textbooks prove that textbook English dialogues "primarily function as reinforcers of the vocabulary students are expected to learn, rather than as models of realistic spontaneous spoken interactions" (Le Foll 2021b: 108). This distinction is even further analysed in her multi-dimensional study of textbook English as a pedagogical variety (Le Foll 2024: 379–467).

2.1.2 Biber's Multi-Dimensional Analysis

The central theoretical basis for this chapter is Biber's (1988) multidimensional account of variation. Biber's core argument is that linguistic features do not occur randomly. Instead, they occur in patterns that fulfil communicative functions. By doing factor analysis on large corpora, Biber identified different dimensions of variation that can be interpreted functionally across spoken and written contexts.

For the present study, Dimension 1 (Involved vs. Informational Production) is especially relevant. On the involved side, language tends to be interactional, context-dependent and person-oriented. On the informational side, language tends to be dense, explicit and noun-heavy. Textbook examples often lean toward informational clarity by design: explicit lexical choices, full clause structures and decontextualised propositions. To communicate appropriately, however, learners need to develop competence in involved usage, especially for informal and conversational settings.

This is one of the key motivations for this study. Can AI be used as a supplement for textbook examples, generating input that closely mirrors real-life conversation in order to familiarise learners with authentic language? If role-prompted LLM output is genuinely sensitive to different registers, the HIGH, NEUTRAL and LOW SCC conditions should show measurable changes in behaviour along involvement-related features. Artificially generated output should therefore not collapse into a textbook-like profile.

2.1.3 Linguistic Markers of Involvement

To make "casualness" more tangible, the present study isolates three concrete markers derived from Biber's register studies (1988; 2021). Rather than recreating Biber's entire multidimensional model, these markers are operationalised as robust indicators that can be compared across different corpora.

The first marker is the density of first- and second-person pronouns (e.g., *I, we, you*). Involved discourse is typically interpersonal: speakers address themselves and their interlocutors directly. Biber (Biber 1988: 105–106) links this feature to interpersonal focus and interactional orientation. In spoken English, person deixis is therefore a structural signal of communicative stance (Biber 2006: 578). Textbook examples tend to minimise this interpersonal discourse in favour of explicit third-person or generic formulations. This design is specifically used to support pedagogical clarity, but may under-represent how participants are addressed in natural conversation. Consequently, a high density of interactive pronouns is treated as a key indicator of involved, casual register.

The second marker is spatial and temporal deictic anchoring: references that depend on the space and time of the context (e.g., *here, there, now, then, tomorrow*). Biber describes these forms as "situation-dependent references" (1988: 115). In practice, these anchors help speakers coordinate attention and action while talking. In contrast, pedagogical examples are frequently designed as context-independent utterances (e.g., generic routines, logical truths). Because of their independence from deixis, these utterances can be taken out of context and still make perfect sense.

The third marker is structural reduction. Language that is produced in spontaneous interaction is regularly compressed to optimise speech economy. This can happen through contractions, ellipses and optional deletions (e.g., complementiser omission) (Biber 1988: 243). Such reductions emerge because conversation relies on shared context, processing economy and speed of the interaction. Conversely, textbooks often suppress reduction in order to teach standard English through maximal transparency. What textbooks tend to ignore are non-standard forms of English that are extremely common in natural conversation. Even though full forms may reduce ambiguity for learners, they also diverge from typical spoken realisations. For this reason, instances of reduction are treated as meaningful indicators for conversational authenticity.

While tracking these markers provides valuable insights into register variation, a comprehensive analysis of pronoun density, deixis and structural reduction falls outside the scope of this thesis and remains open for future research on this topic. Instead, this study approaches the formal/casual level of the English language through an extensive study of phraseology. The theoretical basis for this approach is Sinclair's (1991) distinction between the Open-Choice Principle and the Idiom Principle discussed in Phraseology & Idiomaticity.

2.1.4 Grammatical Mood & Register

Register and mood are two parameters that have to be distinguished clearly. On the one hand, register refers to the situational context shaped by communicative situation, purpose and participants (Biber 1993: 229). On the other hand, mood is a grammatical category on clause level that is used as resource to express modality, speech acts and interpersonal roles through structures such as declaratives, interrogatives and imperatives roles (Halliday and Matthiessen 2014: 134–135). While there is a strong correlation between certain moods and registers (e.g., questions are more commonly used in formal contexts, imperatives in instructional contexts and declaratives in casual contexts), it is important to note that the two parameters are not the same. In other words, register is used on the level of the text as a whole, while mood is a feature that can change from one sentence to another within the same text.

Interrogatives are especially relevant for formal interaction because professional and institutional discourse often relies on questions to request information, clarify understanding and confirm agreement (Biber 2006: 5). In contexts like this, questions are not only used to gather information but also to invoke politeness strategies and manage the flow of an interaction (Brown and Levinson 1978). This is especially visible in utterances using modal verbs (e.g., *Could you please…?* or *Would it be possible…?*) that are mostly used to mitigate the imposition of a request and reduce face-threatening acts, a strategy that is particularly important in formal contexts.

Imperatives are essential to instructional and directive language because they encode the performance of actions directly (Kia 2018: 54). Instructional language tells readers or listeners what to do, in what order and under which conditions (Huddleston and Pullum 2002: 930). Imperatives are the most direct way to express these directives, as they do not require an explicit subject and rely on the shared context to identify the addressee. This makes them especially common in instructional materials, where the learners are tasked with performing specific actions (e.g., *Translate the following sentences.* or *Write a short essay on…*).

Declaratives are the most common mood in casual conversation because they are used to make statements, share everyday information and express opinions (Halliday and Matthiessen 2014: 97). In informal contexts, speakers often use declaratives to share personal experiences, describe their surroundings and express their feelings. They carry personal and emotional content. For example, a sentence like *I really enjoyed the movie we watched yesterday!* is a typical declarative that conveys a personal opinion and an emotion, which is a classic feature of casual conversation.

However, it is important to note that these associations are not absolute. Questions are not inherently formal, imperatives are not exclusively instructional and declaratives can also be used in every other register. Nevertheless, the presence of these moods can be treated as an indicator of the register in which a sentence is produced. For example, a sentence like *Could you please pass the salt?* is more likely to be produced in a formal context, while *Pass me the salt.* is more likely to be produced in an informal context. This pattern shows that variation in situational context and communicative intent is often realised through grammatical choices.

While these syntactic markers provide valuable insights into register variation, they cannot be used as the sole indicator of naturalness. A sentence that is perfectly register-appropriate might still sound unnatural if it does not follow phraseological patterns that are commonly used in actual conversation. Therefore, to evaluate the authenticity of a text, register analysis alone is not sufficient. To get a more complete picture of naturalness, it is necessary to also take a look at phraseology and idiomaticity, transitioning from Biber's register analysis to Sinclair's (1991) distinction between the Open-Choice Principle and the Idiom Principle.

2.2 Phraseology & Idiomaticity

2.2.1 Sinclair's Dual Framework

Register alone cannot fully assess the naturalness of language. Two sentences may display similar register features (e.g., pronoun density, deictic anchoring, structural reduction) but strongly differ in phraseology. To address this issue, this study also incorporates Sinclair's (1991: 109–110) dual framework of language production: the Open-Choice Principle and the Idiom Principle.

Sinclair's contributions mark a significant shift in how we understand language production. Instead of arguing that all parts of language are generated spontaneously at any point, he argues that recurring phraseological patterns function as pre-constructed chunks. These chunks can be accessed as whole units, which allows speakers to communicate more fluently. This insight is especially relevant for pedagogical input, as it suggests that learners need to be exposed to these chunks frequently in order to acquire them correctly. This also implies that grammatical correctness alone is not sufficient for fluency and authentic speech.

2.2.2 The Open-Choice Principle

The Open-Choice Principle assumes that language is created spontaneously word by word. These words are selected to fit the communicative and grammatical constraints of the context at each position in the utterance (Sinclair 1991: 109). This principle is often associated with the idea of infinite creativity in language use, where speakers can combine words in new ways to express new meanings. This model aligns very well with the Grammar-Translation Method (GTM) in language teaching (Benati 2018), which is known for teaching words and grammar in isolation, without paying much attention to how they are used in real-life situations. In the GTM, learners are typically given vocabulary lists and grammar rules, and are expected to apply these rules to form their own sentences. This approach can lead to a lack of exposure to natural phraseology and may thus hinder the development of idiomatic fluency.

Even though the GTM has been proven as ineffective for language acquisition

(Richards 2006), many textbooks still rely heavily on it. This is because the Open-Choice Principle provides maximal transparency for the learners. By teaching vocabulary and grammar in isolation, it allows learners to understand the different building blocks and rules of language without being overwhelmed by the complexity of natural speech. However, removing the ambiguity of the language takes away the opportunity for learners to get familiar with the natural patterns of language use. This issue is especially relevant for high-frequency, polysemous verbs, whose meaning shifts significantly depending on the context. For example, the verb *run* can be used literally (e.g., *I run every morning*) or idiomatically (e.g., *I ran out of money*). If learners are only exposed to the literal use of *run* in the classroom, they may struggle to understand and use the idiomatic meaning. That is why it is important for learners to also be exposed to the idiomatic patterns of language, as Sinclair's Idiom Principle suggests.

2.2.3 The Idiom Principle

The Idiom Principle is a complementary model to the Open-Choice Principle, focusing on the role of pre-constructed chunks in language production. According to this model, speakers have access to a larger pool of fixed and semi-fixed expressions (e.g., idioms, collocations, phrasal verbs) that they can use as whole units, rather than formulating them word by word (Sinclair 1991: 110). This principle aligns with the idea of formulaic language, which plays a crucial role in language acquisition and the production of authentic speech (Schmale 2022: 91–92). Teaching idiomatic and formulaic expressions is one of the key factors of the Communicative Approach (CA) to language teaching (Savignon 1987), which is a counterpart to the GTM. Whereas the GTM focuses on teaching language in isolation, the CA focuses on teaching language in context, with a strong emphasis on communicative competence. Therefore, the CA encourages the use of authentic learning materials that include idiomatic expressions and formulaic language, which can help learners develop a more natural and fluent use of the language.

Corpus-based studies have shown that EFL materials tend to systematically suppress the Idiom Principle in favour of the Open-Choice Principle. The reason why the phraseological profiles of textbook dialogues often deviate from real-world usage is that, "in fact, [. . .] dialogues from teaching manuals are still too often based on the textbook authors' intuitions (Schmale 2004: cf.), which can never yield equivalent results to the study of mass data" (Schmale 2022: 104). For example, chunks that consist of multiple words and highly frequent phrasal verbs are heavily under-represented in EFL textbook dialogues (Le Foll 2024: 72).

This distinction is the key motivator for the design of the present study. If on the one hand, textbooks are designed to maximise clarity and transparency, they should therefore show a strong bias toward the Open-Choice Principle, with high usage rates of literal contexts in comparison to idiomatic contexts. On the other hand, if LLMs can truly imitate authentic language, they should comply to the Idiom Principle and naturally produce output with a higher proportion of idiomatic contexts, especially in the HIGH and LOW register conditions. Furthermore, if the SCC output is genuinely more sensitive to phraseological patterns, patterns of idiomaticity should not be the same across all verbs, but show lemma-specific variation that reflects the natural distribution of literal and idiomatic use cases.

2.2.4 De-lexicalisation in High-Frequency Verbs

A key concept related to the Idiom Principle is de-lexicalisation, which refers to the process where a verb loses its original, concrete meaning and becomes part of a fixed expression or idiom, which may bear little to no relation to its literal counterpart. Sinclair defines this process as a "reduction of the distinctive contribution made by that word to the meaning" (Sinclair 1991: 113). Because these verbs lose their independent meaning, they heavily rely on the context they are used in, indicating that "normal text is largely de-lexicalised, and appears to be formed by exercise of the idiom principle" (Sinclair 1991: 113). This is especially relevant for high-frequency, polysemous verbs. Because of their high dependence on context, they are very suitable for a direct comparison in the NRC and SCC.

While processing de-lexicalised chunks is crucial for developing idiomatic fluency, their lack of transparency can be a significant obstacle for learners. This is a burden that increases the cognitive load (Sweller 1988) of the input, which can be overwhelming when learning a new language.

2.3 Pedagogical Input & Cognitive Complexity

2.3.1 Krashen's Input Hypothesis ($i + 1$)

Krashen's Input Hypothesis (1985) provides a pedagogical perspective on the quality of input. This hypothesis proposes that acquisition occurs when learners are exposed to input that is largely comprehensible at their current level of competence (i), while still containing some elements that are just beyond their current level ($+1$). This means that the input should be challenging, but not too overwhelming. A perfect sentence for a learner would therefore contain mostly familiar vocabulary and grammar, while also introducing new words or structures that the learner can understand through context. The implication for this study is simple: example sentences should not only be judged by correctness, but also whether they provide useful input that can be processed and acquired by learners. On the one hand, if the sentences are too complex (e.g., too many unfamiliar de-lexicalised chunks), they may not be comprehensible through context, which hinders acquisition. On the other hand, if the sentences are too simple (e.g., only known words and structures), they cannot provide the necessary challenge to push learners to acquire new features of the language. In this thesis, the B1/B2 level of the Common European Framework of Reference for Languages (CEFR) is used as a benchmark, where the transition from transparent, literal syntax to more complex, idiomatic patterns is the most relevant hurdle to overcome.

Taking this hypothesis into account, there are two potential risks that emerge. First, if the textbook sentences are indeed over-sanitised and lack idiomaticity, they may remain at i level, lacking exposure to the $+1$ level of natural phraseology and register variation. As Krashen warns, "over-simplification can delay acquisition by denying the acquirer $i+1$" (Krashen 1985: 24). Second, if synthetic input introduces too many new concepts and de-lexicalised chunks without sufficient context, it risks "excess $i+n$, rules beyond the acquirer's $i+1$" (Krashen 1985: 24). This excess can be described as incomprehensible "noise", making the sentences unusable for learners. Therefore, the practical goal of the AI-generated sentences should not be achieving maximal colloquiality, but rather producing what Krashen describes as "the best data [...] which contains maximum richness but which remains comprehensible" (Krashen 1985: 27). By finding this balance between naturalness and comprehensibility, the AI-generated sentences may provide the optimal $i+1$ challenge for learners at B1/B2 level.

2.3.2 Sweller's Cognitive Load Theory

Sweller's Cognitive Load Theory (1988) provides a different perspective on the quality of input. According to this theory, "the system (or person) has a fixed cognitive capacity" and "both problem solving and learning require some of that capacity" (Sweller 1988: 275). Learning tasks demand cognitive resources, because "human short-term memory is severely limited" and being exposed to unnecessary complexity in learning tasks may "contribute to an excessive cognitive load" (Sweller 1988: 265). These demands on cognitive resources can be classified as either intrinsic load (the inherent complexity of the content) or extraneous load (the way the content is presented). Because the working memory is so limited, even "a small additional load could be critical", meaning that "any increase in resources required [...] must inevitably decrease resources available for learning" (Sweller 1988: 275).

This finding is especially relevant for pedagogical sentences, showing that the same sentence can be either supporting or hindering how learners process and acquire the language. Awkward phrasing, unnatural flow of information or poor lexical choices can all contribute to what Sweller describes as "cognitive overload which leaves little capacity for other aspects of the tasks" (Sweller 1988: 276). Le Foll also acknowledges this and argues that the danger of this overload is exactly why "it may make pedagogical sense to adapt authentic texts for specific proficiency levels" (Le Foll 2024: 219). However, phrases that are already familiar can in contrast also reduce the cognitive load through predictability, which may free up capacity for processing new information. In Sweller's framework, pre-constructed chunks and formulaic expressions are stored in the long-term memory as "schemas" that can be accessed as whole units without using up cognitive resources in the working memory. When applied to Sinclair's dual framework, these expressions function as such cognitive schemas. Rather than exhausting the working memory with the need to process each word individually (Open-Choice Principle), learners can access these chunks as whole units (Idiom Principle), without overloading their cognitive resources. Therefore, being exposed to idiomatic chunks is not only important for developing natural speech, but also efficient for freeing up working memory capacity to process new $+1$ level concepts. To determine if AI can replicate these pedagogically useful chunks, it is necessary to look into the linguistic architecture of LLMs.

2.4 The Linguistic Architecture of Large Language Models

2.4.1 Next-Token Prediction & Distributional Semantics

At their core, LLMs function on a rather simple mechanism: the prediction of likely sequences of words based on context. This is known as Next-Token Prediction (NTP). Shanahan describes LLMs as "generative mathematical models of the statistical distribution of tokens in the vast public corpus of human-generated text" (2022:

2). This means that LLMs are trained on large datasets of different texts, making it possible for them to learn statistical patterns of language use. By learning these patterns, LLMs can generate content that is statistically likely to occur within a given context. Therefore, when presented with a prompt, an LLM calculates “what words are most likely to follow the sequence” based on its patterns in the texts it has been trained on (Shanahan 2022: 2). Even though this NTP process is only based on mathematical probabilities, it can produce output that is surprisingly coherent, contextually relevant and bears semantic meaning (Zhao and Thrampoulidis 2025).

In Natural Language Processing (NLP), profiles for words are created based on “how often it appears next to another word” (Holmes et al. 2019: 214). In their study, Zhao et al. (Zhao and Thrampoulidis 2025) demonstrate that this aligns perfectly with the theory of distributional semantics (Harris 1954): by optimising the model for NTP, LLMs effectively learn “context-to-next-token co-occurrence patterns” (Zhao and Thrampoulidis 2025: 1). Consequently, this means that “words appearing in similar contexts have high similarity” (Zhao and Thrampoulidis 2025: Section 3.1), which is what distributional semantics is all about.

2.4.2 Attention Mechanisms & Contextual Sensitivity

The core component that allows LLMs to generate contextually relevant output is the attention mechanism, which relies on the transformer architecture of a model. For models like ChatGPT or Gemini, the concept of self-attention is crucial. Unlike earlier models that used to process the input sequentially, the transformer architecture using self-attention allows for parallel processing of the input, meaning that the relationships between all elements in a sequence can be assessed simultaneously (Vaswani et al. 2017). This attention mechanism is what allows AI-powered chatbots to generate output that is contextually relevant to the input of the user.

This is not only relevant for machines, but also has implication for human interaction. It functions basically as a computational equivalent to Biber’s Register Theory. Biber argues that human language varies based on “situational characteristics of the speaker and addressee, and the social role relationship” (Biber 1988: 38), which is exactly what the attention mechanism does: it takes into account the situational characteristics of the input (e.g., the prompt) and generates output that is relevant to those characteristics. Whereas register in human communication relies on the situational context, attention to register in LLMs relies on the input prompt of the user. For instance, if the prompt specifically asks for a workplace setting (the HIGH register condition), versus a casual conversation (the LOW register condition), the self-attention mechanism should weigh contextual tokens like 'formal' or 'casual' mathematically through "judicious use of prompt prefixes" (Shanahan 2022: 4). This mathematical weighting forces the model to rethink its choices of vocabulary and syntax. The texts LLMs are trained on are naturally "related by their exploitations of those [contextual] functions" (Biber 1988: 36), which allows the model to generate output that is sensitive to specific registers asked for in a prompt. Consequently, the model should adapt the phraseological patterns such as de-lexicalised chunks to match the communicative context the prompt is asking for. This means that the prompt itself plays a crucial role in the functionality of the model, as it guides the attention mechanism and therefore the output of the model.

2.4.3 Reinforcement Learning from Human Feedback & Prompt Engineering

Even though raw models have complex statistical architectures that allow them to generate impressive output, they are rarely deployed without further fine-tuning. Because raw models are "simply using patterns learned from [their] training data to predict the next word(s)", they "have little ability to understand user intent" and are merely "appending text" to a prompt (Bergmann 2023). This is where Reinforcement Learning from Human Feedback (RLHF) comes into play. RLHF is a machine learning technique that "translate[s] human preference into a numerical reward signal" (Bergmann 2023). By using systems like the Proximal Policy Optimization (PPO) algorithm, the model is continuously updated to "optimi[s]e a policy to yield maximum reward" (Bergmann 2023) based on this human feedback. Shanahan notes that this fine-tuning process is crucial to "mould a model's responses to better reflect user norms" (Shanahan 2022: 10). Ultimately, RLHF is what allows the model to go from raw statistical token prediction to a more user-friendly and helpful AI assistant.

However, this process, even though it is essential for the model's capability to understand the user's intent, brings a potential pedagogical paradox with it. Research into RLHF has shown that giving specific instructions on politeness and clarity can unintentionally suppress the model's linguistic versatility and creativity, a phenomenon often described as "alignment-tax" (Ouyang et al. 2022). This loss of diversity is even more problematic when combined with "reward hacking", which happens when the model learns to exploit the reward system, overproducing the exact same linguistic features that human users tend to reward (e.g., politeness, clarity) (Amodei et al. 2016). This means that the model may end up producing universally formal, neutral and cautious output to maximise the reward, which is exactly what the present study is trying to avoid. To mitigate this issue, precise prompt engineering is necessary.

Through prompt engineering, users can explicitly define the communicative context and the desired linguistic features of the generated output. For example, when asking the model to generate sentences in a casual, informal tone (LOW), it forces the transformer architecture to weigh the tokens related to informality more heavily. This targeted weighting can help to counteract the alignment tax and reward hacking. It compels the model to reach deeper into its pre-training data to find semantic and syntactic patterns that are more closely aligned to the desired answer, rather than defaulting to commonly rewarded patterns. Prompt engineering is therefore an essential step to mitigate the alignment tax, pushing the model toward the Idiom Principle and generate diverse output that is usable for B1/B2 level English learners.

2.5 Synthesis: Evaluating Synthetic Input

Taken together, the framework consists of five different perspectives for generating and evaluating synthetic pedagogical texts. First, Biber's register theory offers a foundational understanding of how language varies based on situational context. Second, Sinclair's dual framework of the Open-Choice Principle and the Idiom Principle provides a model for idiomaticity and authenticity of language. Third, Krashen's Input Hypothesis establishes a pedagogical perspective on the quality of input and the balance between comprehensibility and challenge. Fourth, Sweller's Cognitive Load Theory proposes a different view on the quality of input, focusing on the cognitive resources required to process the input. Finally, the linguistic architecture of LLMs, especially the attention principle and the role of prompt engineering, provides a technical perspective and a look behind the scenes of how AI-generated output is produced and how it can be guided to meet specific standards. Table 1 summarises the theoretical perspectives and shows how they are used in the empirical analysis of this study.

Table 1: Theoretical Framework Summary

Dimension	Theoretical Basis	Theoretical Indicator
Register[1]	Biber (1988)	Pronouns, deixis, contractions
Phraseology	Sinclair (1991)	`LITERAL` vs. `IDIOMATIC`
Pedagogy	Krashen (1985)	B1/B2 level appropriateness
Processing[1]	Sweller (1988)	Cognitive Load
Architecture	Vaswani et al. (2017)	Self-Attention & Prompt Engineering

This synthesis of perspectives directly motivates the design choices of the methodology in Methodology, where this theoretical framework is used as basis for the empirical analysis of this study.

3 Methodology

This section explains how the study was carried out: the research design, the corpus compilation process, the data processing pipeline, the statistical analysis and ethical considerations. To make the workflow transparent and reproducible, the methodology is described step-by-step, with detailed specifications and requirements for each step of the way. All corresponding datasets, extraction scripts and analytical code are provided in Appendix D. The overall workflow is visualised in Figure 1.

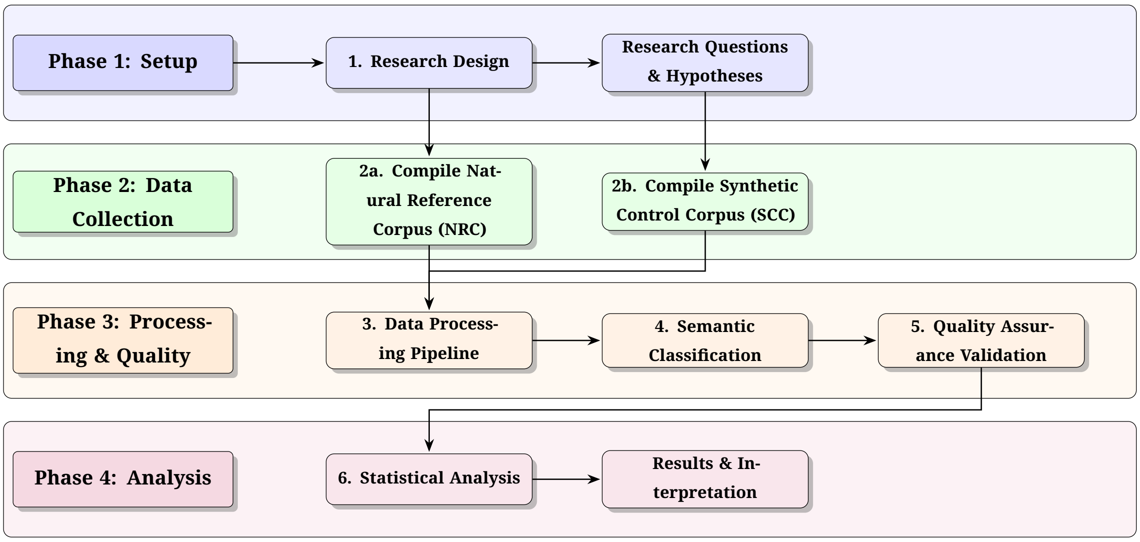


Figure 1: Overview of the empirical workflow, structured into four main phases: Setup, Data Collection, Processing & Quality and Analysis.

[1]While these indicators define the theoretical boundaries of the constructs, the present empirical pipeline (Methodology) restricts its quantitative operationalisation exclusively to the phraseological dimension.

3.1 Research Design & Objectives

3.1.1 Study Design Overview

This study is designed as a comparative corpus-based analysis with two datasets: a Natural Reference Corpus (NRC) based on human-written sentences from EFL textbooks, and a Synthetic Control Corpus (SCC) generated by the Gemini 2.5 Flash model under controlled prompt conditions. These conditions are namely register specifications. Practically, this study follows an approach with different methods, combining quantitative and qualitative elements to make statistically relevant comparisons while ensuring the validity of the data.

The target of analysis is the phraseological behaviour of high-frequency, polysemous verbs in the NRC and SCC. The NRC serves as a benchmark of authentic pedagogical language against which machine-generated output can be compared, while the nature of the SCC allows for direct testing of the model's abilities through prompt engineering. The focus is on the individual target verbs at sentence level. Each occurrence in both corpora is classified as either `LITERAL` or `IDIOMATIC` based on its semantic context, which allows for a direct comparison of the phraseological profiles of the two corpora.

3.1.2 Research Question & Hypotheses

The core research question asks: *To what extent can Large Language Models (specifically Gemini 2.5 Flash) generate linguistically distinct registers (formal, instructional and casual) that quantitatively differ from human-authored textbook materials at the level of phraseological behaviour?*

This question is tested through three verifiable hypotheses:

- **H1 (Pedagogical Simplification):** The NRC will show a higher proportion of literal uses than the SCC overall, reflecting pedagogical preference for Open-Choice transparency over Idiom-Principle patterning.
- **H2 (Corpus-Informed Pedagogy):** The NRC will pattern most closely with SCC-NEUTRAL, indicating that textbook English functions as a distinct instructional register rather than approximating formal or casual native discourse.
- **H3 (Controlled Idiomaticity):** The NRC will show selective idiomaticity (non-uniform idiomatic behaviour across verb types) rather than uniformly high or uniformly low idiomaticity, reflecting strategic rather than blanket phraseo-

logical simplification.

3.2 Corpus Compilation

3.2.1 Lemma Selection

Chapter 2 established that high-frequency, polysemous verbs are particularly relevant for English learners at the B1/B2 level for various reasons. They are susceptible to de-lexicalisation, which makes their meaning dependent on the context they are used in, disguising their lexical meaning. This makes them ideal for testing the phraseological capabilities of LLMs. Therefore, the sentence-level comparison in this study focuses on the following ten target verb lemmas: RUN, TAKE, MAKE, KEEP, HOLD, BREAK, DROP, TURN, SEE and LOOK.

As Le Foll (2024) highlights, these verbs present a significant challenge for learners due to their de-lexicalised nature and high frequency in both literal and idiomatic contexts. In her analysis of EFL exercises designed for learning new vocabulary, she specifically isolated the lemmas MAKE and TAKE as examples of verbs carrying this burden: "[...] all the instances of the lemmas MAKE and TAKE were automatically retrieved from the vocabulary exercise subcorpora and the results were manually sorted for meaning and collocational patterns. High-frequency verbs are found to feature prominently in the context of restricted collocations in all the textbooks" (Le Foll 2024: 46). Extending upon this selection, the present study includes eight additional high-frequency, polysemous verbs that are commonly used in both literal and idiomatic contexts. These core English verbs are typically among the first verbs that learners encounter in their language learning journey (A1/A2). As Huddleston and Pullum observe, such "common verbs like *bring, come, give, go, have, let, make, put, take*, etc., are found in large numbers of [...] idioms, often with a considerable range of meanings" (2002: 284). Tracking how this isolated set of verbs behaves across `LITERAL` and `IDIOMATIC` contexts in the NRC and SCC provides a direct metric for evaluating the phraseological capabilities of the model as well as the degree of simplification in the textbooks.

To ensure consistency, the selected lemmas were compiled and stored in a `lemmas.txt` file. This file served as an immutable reference for the following data processing steps. The full list of lemmas is documented in Appendix D.1.

3.2.2 Natural Reference Corpus

The NRC was drawn from the English Textbook Corpus (TEC) compiled by Le Foll (2021a). This corpus is derived from a selection of EFL textbooks commonly used in European classrooms, covering the levels from A1 up to B2. The specific textbooks that were used are documented in Appendix A. Subsequently, TEC refers to the original source corpus, while NRC refers to the specific subcorpus that was extracted for the present study. This subcorpus was extracted from the TEC through a process with multiple stages of filtering and annotation:

1. **CEFR-Level Filtering:** The TEC was first filtered to include only sentences from the B1 and B2 levels, which are the most relevant for the target learner group. This was done through the metadata tagging in Sketch Engine, which allows for filtering based on CEFR level.

2. **Lemma-Filtering:** The B1/B2 subcorpus was further thinned out to include only sentences containing at least one of the target verb lemmas. This was done through the `create subcorpus` feature in Sketch Engine, which allows for the creation of subcorpora based on specific search criteria.

3. **Sketch Engine Export:** The resulting subcorpus was exported from Sketch Engine in eXtensible Markup Language (XML) format, which preserves the original sentence structure and metadata.

4. **Conversion to JSON/CSV:** The XML files were then converted to JavaScript Object Notation (JSON) and Comma-Separated Values (CSV) formats for easier processing in Python, which is the main programming language used for the data processing pipeline.

5. **NLTK-Tagging & -Filtering:** The sentences were processed using the Natural Language Toolkit (NLTK) package in Python to perform part-of-speech (POS) tagging. This method was used to verify that the target verbs were indeed used as verbs in the sentences, rather than as nouns or other parts of speech. Other incidences of the target lemmas that were not tagged as verbs were filtered out at this stage. This step was necessary to ensure that the analysis focuses on the verb usage of the target lemmas, since other parts of speech may not be of phraseological nature and thus irrelevant for the research question. The full script for this process is provided in Appendix D.2

6. **Manual Validation:** A random sample of sentences was manually checked

to ensure that the POS-tagging and filtering were accurate. Any errors found in this sample were used to refine the tagging and filtering process before applying it to the entire dataset. The distribution of the target verb lemmas in the NRC after POS filtering is shown in Table 2.

Table 2: Distribution of target verb lemmas in the NRC dataset after POS filtering

Lemma	**n**	**%**
run	218	3.96
take	1,067	19.40
make	1,493	27.14
keep	244	4.44
hold	145	2.64
break	137	2.49
drop	32	0.58
turn	290	5.27
see	1,030	18.72
look	845	15.36
Total	**5,501**	**100.00**

Note: After POS filtering, the NRC yielded ($N = 5{,}501$). However, this table reports pre-semantic-classification counts. Following semantic classification, 18 unresolved items were excluded, yielding the final analysis sample of $N = 5{,}483$ reported in Chapter 4.

3.2.3 Synthetic Control Corpus

The SCC was entirely generated by the Gemini 2.5 Flash model, using its Application Programming Interface (API). This model was chosen for its stable API access, fast processing speed and practical cost in high-volume data generation. The sentences in the SCC were generated through a process of prompt engineering, which is described in detail in the next section. The target of generation was set to 150 sentences per lemma, with 50 sentences for each of the three register conditions (HIGH, LOW, NEUTRAL). This resulted in a total of $N = 1{,}500$ sentences in the SCC dataset, of which $N = 1{,}470$ could successfully be parsed from the generation output.

To maintain methodological consistency with the NRC, this dataset was also subjected to the same NLTK-based filtering and classification processes to filter out non-verb occurrences (detailed in Section 3.3.1). The final SCC yielded $N = 1{,}237$ sentences (HIGH: 427; NEUTRAL: 385; LOW: 425). The imbalance in the distribution of the sentences across the three registers reflects different rates of non-verbal use of the target lemmas across the registers, which is an expected outcome of the generation process.

3.2.4 Pilot Study & Prompt Refinement

Before executing the full data generation process for the SCC, a pilot study was conducted. This pilot study was designed to identify potential issues with the prompt engineering, instruction ambiguity and failure rates of the model's API. This process required several changes to the original prompt design to ensure the LLM could reliably produce outputs that met the specific requirements of the study.

To test the API's reliability, first, a small debug script (Appendix D.3) was used to verify successful API calls and to check for any potential issues with rate limits. After confirming that the API was functioning correctly, a naive, zero-shot prompt was drafted and tested:

> *"Write a B2-level English sentence using the verb 'take' in an idiomatic context".*

However, the outputs revealed that the prompt failed to produce the desired results. The model frequently included conversational fillers, such as "Sure!" or "Here's an example:", which broke the automated data extraction process. Furthermore, the sentences were mostly structurally similar, revealing that the model was reward hacking by overproducing the same patterns of output. Additionally, the model often failed to meet the CEFR level requirement, occasionally hallucinating and producing sentences that were either too simple (A1/A2) or too complex (C1/C2).

To tackle these structural issues, the prompt was turned into a programmatic template to enforce a specific output structure. While demanding a strict matrix of register, mood and context, the prompt asked for an output in the form of a JSON object with keys for each required element. This structured demand ended up producing cleaner output that could be easily parsed and extracted for the analysis. To mitigate the reward hacking issue, the prompt was further refined to include specific instructions to avoid repetition:

> SYSTEM PROMPT:
> *"You are a linguistic expert specialising in pragmatics and corpus generation. Your task is to generate distinct, high-quality sentences for a target word based on strict register constraints. The sentences must be appropriate for a CEFR B1/B2 learner."*
> PROMPT TEMPLATE:
> *"Target Word: '`{word}`'*
> *Batch ID: `{batch_num}` (Ensure these sentences are unique from generic examples)*
>
> *Generate a JSON array containing exactly `{n}` sentences:*
> *1. `{n}` sentences: Register HIGH (Formal), Mood Question. Context: Workplace/Professional.*
> *2. `{n}` sentences: Register LOW (Casual), Mood Statement. Context: Friends/Personal/Slang.*

3. `{n}` sentences: Register NEUTRAL (Direct), Mood Imperative. Context: Instructions/Navigation/Rules.

Vary the sentence structure and vocabulary usage. Output JSON keys: 'register', 'mood', 'sentence'."

After implementing this refined prompt, a second pilot run was conducted. This run was executed across multiple target lemmas to verify that the new prompt successfully resolved the issues identified in the first pilot. The results showed a significant improvement in quality and diversity of the generated sentences, CEFR level compliance and the absence of conversational fillers. The sentences generated in the second pilot run were then used to test the data extraction and processing pipeline. The preliminary results from this test run, as well as distributions of the target verb lemmas and visualisations of their behaviour across the different registers and moods, are documented for transparency in Appendix B. The Appendix also includes short interpretations of the pilot results.

3.3 Data Processing Pipeline

After the compilation of the NRC and SCC datasets, both datasets were processed through a staged annotation pipeline to prepare them for the final analysis. This pipeline consists of several steps, including part-of-speech (POS) tagging, syntactic disambiguation, semantic classification, data quality assurance (QA) and inter-rater reliability (IRR) assessment.

3.3.1 Part-of-Speech-Tagging & Syntactic Disambiguation

Because the English language relies heavily on zero-derivation, many words can function as different parts of speech depending on the context. For example, the word TAKE can function as a verb in the sentence "I take the book", but it can also be used as a noun in forms like "I have a different take on this book". If uses of the target lemmas that are not tagged as verbs were included in the analysis, the LLM judge would be forced into making choices based on the syntactic context rather than the semantic context, potentially contaminating the dataset with an unrealistically high number of `LITERAL` or invalid contexts.

To prevent this issue, the Natural Language Toolkit (NLTK) (Bird et al. 2009; Loper and Bird 2002) package in Python was used to perform POS-tagging on both the NRC and SCC. This process uses Natural Language Processing (NLP) techniques to classify each word in the sentences according to its part of speech. The NLTK's POS-tagger was trained on the Penn Treebank corpus, which is a widely used annotated corpus of English, meaning that it has a high level of accuracy and can thus be reliably used for this task. It is also a free-to-use library, which makes it accessible and reproducible for other researchers.

The verb filtering was done using two custom Python scripts tailored to each corpus's data format. For the SCC, the `pos_taggerV2NLTK.py` (see Appendix D.5) script processed the data from JSON format, filtering the sentences and extracting only those containing the target verbs. For the NRC, the `pos_taggerNLTK_forCSV.py` (see Appendix D.6) script performed the same verb filtering operation but processed CSV input directly, ensuring methodological consistency, the only difference being the different data source formats.

In detail, the scripts work as follows: first, the `word_tokenize` module is used to split each sentence into individual words. Then, the `pos_tag` function assigns a part-of-speech tag to each word. A sentence is only retained if it contains at least one token tagged as a verb (e.g., VB, VBD, VBG, VBN, VBP, VBZ) whose lemmatised form matches the target lemma. Next, the script checks the verb tokens with the `WordNetLemmatizer`, which reduces inflected forms to their base form. These steps ensure that only sentences where the target lemmas are used as verbs are included in the final datasets.

This POS filter proved its worth, as it successfully filtered out 660 sentences from the NRC (10.7%) leaving a clean dataset yielding $N = 5{,}501$ sentences. In the SCC, 233 sentences (15.9%) were filtered out, leaving $N = 1{,}237$ sentences for the next step: semantic classification.

3.3.2 Semantic Classification

The semantic classification was carried out through an LLM-as-a-judge approach (Zheng et al. 2023). Instead of creating more text, here, the LLM was used to evaluate the existing sentences and classify them as either `LITERAL` or `IDIOMATIC` based on their semantic context.

Existing studies tend to use Key Words in Context (KWIC) of the sentences to make the classification task easier for human annotators, but this approach only uses a small part of a sentence (context window), which may not contain enough information to make an accurate classification. Idiomaticity and de-lexicalisation processes often rely on the broader context or long-distance syntactic dependencies of a sentence (e.g., preceding subordinate clauses like in "Given the evidence, the *position* that the defence attorney ultimately chose to *take* was unsurprising.") making a classification based on a context window of just a few words unreliable.

The LLM-as-a-judge approach allows for the classification across the entire sentence, no matter how long or complex it is. This makes it not only easier for the annotator, but also more accurate, as it can take all relevant information of the sentence into account before deciding on the classification. The full prompt used for the classification is provided in Appendix E.2. It forced the LLM to make a strict, binary choice between `LITERAL` and `IDIOMATIC` for each sentence based on the semantic context, returning the classification in a structured JSON array that can easily be parsed for the next steps of the analysis.

3.3.3 Data Quality Assurance

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like "Huardest gefburn"? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

3.3.4 Inter-Rater Reliability Assessment

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like "Huardest gefburn"? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

The full agreement distribution is shown in Table 3.

		AI Model (Gemini)		
		LITERAL	**IDIOMATIC**	**Total**
Human Rater	**LITERAL**	62	20	82
	IDIOMATIC	15	89	104
	Total	77	109	**186**

Table 3: Confusion Matrix of Semantic Classifications for the IRR sample ($N = 186$). The diagonal cells represent raw agreement (81.2%), while the off-diagonal cells represent disagreements penalized by the Cohen's Kappa calculation ($\kappa = 0.62$).

3.4 Statistical Analysis

3.4.1 Comparison Strategy

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like "Huardest gefburn"? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

3.4.2 Inferential Testing & Thresholds

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like "Huardest gefburn"? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

3.4.3 Statistical Procedure & Frequency Normalisation

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like "Huardest gefburn"? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

3.5 Ethical Considerations & Data Use

3.5.1 Copyright & Data Distribution

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

3.5.2 API Privacy & Data Security

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

3.5.3 Sociolinguistic Bias in AI-Generated Content

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

4 Results

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like "Huardest gefburn"? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

4.1 Descriptive Overview of the Final Datasets

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like "Huardest gefburn"? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Table 4: Descriptive overview of final annotated datasets

Condition	**N**	**LITERAL n (%)**	**IDIOMATIC n (%)**
NRC	5,483	2,503 (45.7)	2,980 (54.3)
SCC-HIGH	427	56 (13.1)	371 (86.9)
SCC-NEUTRAL	385	197 (51.2)	188 (48.8)
SCC-LOW	425	81 (19.1)	344 (80.9)
SCC (combined)	1,237	334 (27.0)	903 (73.0)
Combined total	6,720	2,837 (42.2)	3,883 (57.8)

4.2 Overall Corpus Comparison (Testing Hypothesis 1)

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like "Huardest gefburn"? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Table 5: Overall NRC vs. SCC contingency table (Corpus $\times$ Usage Category)

Corpus	**LITERAL n (%)**	**IDIOMATIC n (%)**	**Total**
NRC	2,503 (45.7)	2,980 (54.3)	5,483
SCC (combined)	334 (27.0)	903 (73.0)	1,237

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

4.3 Register Effects & Prompt Sensitivity (Testing Hypothesis 2)

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Table 6: Pairwise NRC vs. SCC register comparisons

Comparison	**Idiomatic rate (%)**	$\chi^2(1)$ **(**p**)**	**Cramér's** V
NRC vs. HIGH	54.3 vs. 86.9	170.81 ($p < 0.001$)	0.170
NRC vs. LOW	54.3 vs. 80.9	113.34 ($p < 0.001$)	0.138
NRC vs. NEUTRAL	54.3 vs. 48.8	4.41 ($p = 0.036$)	0.024

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like "Huardest gefburn"? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

4.4 Lemma-Level Selectivity (Testing Hypothesis 3)

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like "Huardest gefburn"? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Table 7: Lemma-level idiomaticity across NRC and SCC conditions (% `IDIOMATIC`)

Lemma	**NRC**	**HIGH**	**NEUTRAL**	**LOW**
break	56.9	100.0	52.5	85.0
drop	40.6	95.5	17.4	81.4
hold	53.1	87.8	19.1	76.0
keep	56.6	34.7	28.9	59.2
look	48.5	90.7	42.9	90.0
make	58.3	90.0	58.3	84.0
run	35.0	100.0	81.0	78.0
see	33.6	91.8	74.0	82.1
take	75.7	95.9	58.3	92.0
turn	58.6	97.1	34.6	88.2

Note: SCC percentages are calculated from syntactically filtered cell sizes (see Part-of-Speech-Tagging & Syntactic Disambiguation). NRC sample sizes vary reflecting natural frequency.

5 Discussion

5.1 Summary of Key Findings

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

5.2 Interpreting the Hypotheses

5.2.1 Interpreting H1: The Pedagogical Simplification Hypothesis

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

5.2.2 Interpreting H2: Textbooks as Instructional Register

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

5.2.3 Interpreting H3: Lemma-Level Selectivity

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

5.3 The V-Shaped Pattern: Register Sensitivity in Large Language Models

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

5.4 Methodological Reflections

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

5.5 Limitations

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

5.6 Future Research

5.6.1 Regulating Lexical Boundaries for Optimal $i + 1$ Input

A highly promising avenue for future research involves computationally regulating the lexical boundaries of synthetic input to perfectly align with Krashen’s i+1 hypothesis. While the present study relies on general CEFR B1/B2 prompting, future methodologies could employ strict systematic prompting or Retrieval-Augmented Generation (RAG) to restrict an LLM’s vocabulary output exclusively to the lexical items introduced up to a specific chapter in a target textbook. By firmly anchoring the base vocabulary at the learner’s exact current competence (i), the model could be tasked with generating authentic, register-appropriate sentences that introduce complexity solely through novel phraseological combinations and de-lexicalised chunks (+1). This pedagogical constraint would theoretically prevent the AI from overwhelming the learner’s working memory with unknown open-choice vocabulary, isolating the cognitive load entirely to the acquisition of natural, context-dependent idiomaticity.

6 Conclusion

6.1 Research Summary

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

6.2 Theoretical Contributions

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

6.3 Practical Implications

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

6.4 Final Reflection

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

References

Amodei, Dario, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané (2016). *Concrete Problems in AI Safety*. DOI: 10.48550/arXiv.1606.06565. arXiv: 1606.06565 [cs]. (Visited on 04/27/2026).

Benati, Alessandro (2018). "Grammar-translation method". *The TESOL encyclopedia of English language teaching*, pp. 1–5.

Bergmann, Dave (2023). *What Is Reinforcement Learning From Human Feedback (RLHF)? | IBM*. https://www.ibm.com/think/topics/rlhf. (Visited on 04/27/2026).

Biber, Douglas (1988). *Variation across Speech and Writing*. 1st ed. Cambridge University Press. ISBN: 978-0-521-36543-7. DOI: 10.1017/CBO9780511621024.

Biber, Douglas (1993). "Using Register-Diversified Corpora for General Language Studies". *Computational Linguistics* 19.2. Ed. by Julia Hirschberg, pp. 219–241. URL: https://aclanthology.org/J93-2001/.

Biber, Douglas (2006). *University Language: A Corpus-Based Study of Spoken and Written Registers*. Vol. 23. (Studies in Corpus Linguistics). Amsterdam: John Benjamins Publishing Company. ISBN: 978-90-272-2295-4 978-90-272-2296-1 978-90-272-9362-6. DOI: 10.1075/scl.23. (Visited on 04/16/2026).

Biber, Douglas, Stig Johansson, Geoffrey N. Leech, Susan Conrad, and Edward Finegan (2021). *Grammar of Spoken and Written English*. Amsterdam: John Benjamins Publishing Company. ISBN: 978-90-272-0796-8 978-90-272-6047-5. DOI: 10.1075/z.232. (Visited on 12/26/2025).

Bird, Steven, Ewan Klein, and Edward Loper (2009). *Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit*. Sebastopol, CA: O'Reilly Media. ISBN: 9780596516499.

Brown, Penelope and Stephen C. Levinson (1978). *Questions and Politeness : Strategies in Social Interaction*. Cambridge ; New York : Cambridge University Press. ISBN: 978-0-521-21749-1. (Visited on 04/20/2026).

Conti, Gianfranco and Steve Smith (2016). *The ugly truth about school-based Modern Language teaching*. Critiques unnatural textbook approaches vs. natural language acquisition. URL: https://gianfrancoconti.com/2016/11/26/the-ugly-truth-about-school-based-modern-language-teaching/.

Curdt-Christiansen, Xiao Lan and Csilla Weninger, eds. (2015). *Language, Ideology and Education: The Politics of Textbooks in Language Education.* (Routledge Research in Language Education). Hoboken: Taylor and Francis. ISBN: 978-1-317-80385-0 978-0-415-84038-5.

Halliday, M. A. K. and Christian M. I. M. Matthiessen (2014). *Halliday's Introduction to Functional Grammar*. 4th ed. London: Routledge.

Harris, Zellig S (1954). "Distributional structure". *Word* 10.2-3, pp. 146–162.

Holmes, Wayne, Maya Bialik, and Charles Fadel (2019). *Artificial Intelligence in Education: Promises and Implications for Teaching and Learning*. Boston, MA: Center for Curriculum Redesign. ISBN: 978-1-7942-9370-0.

Huddleston, Rodney and Geoffrey K. Pullum (2002). *The Cambridge Grammar of the English Language*. Cambridge: Cambridge University Press. ISBN: 978-0-521-43146-0.

Kia, Elnaz (2018). "Directive use in university classroom discourse: Variation across disciplines, academic levels, and interactivity". PhD thesis. Northern Arizona University.

Krashen, Stephen D. (1985). *The Input Hypothesis: Issues and Implications*. London; New York: Longman. ISBN: 978-0-582-55381-1.

Le Foll, Elen (2021a). *Bibliographic Metadata of the Textbook English Corpus (TEC). Textbook English: A Multi-Dimensional Approach, v. 1.1*. DOI: 10.5281/zenodo.4922819. (Visited on 01/31/2026).

Le Foll, Elen (2021b). "Register variation in school EFL textbooks". *Register Studies* 3.2, pp. 207–246. DOI: 10.1075/rs.20009.lef. (Visited on 01/31/2026).

Le Foll, Elen (2024). *Textbook English: A Multi-Dimensional Approach.* (Studies in Corpus Linguistics (SCL)) volume 116. Amsterdam Philadelphia: John Benjamins Publishing Company. ISBN: 978-90-272-4680-6. DOI: gruy-9789027246806.

Loper, Edward and Steven Bird (2002). "NLTK: The Natural Language Toolkit". In: *Proceedings of the ACL-02 Workshop on Effective Tools and Methodologies for Teaching Natural Language Processing and Computational Linguistics*. Philadelphia, Pennsylvania, USA: Association for Computational Linguistics, pp. 63–70. DOI: 10.3115/1118108.1118117.

Ouyang, Long, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter

Welinder, Paul Christiano, Jan Leike, and Ryan Lowe (2022). *Training Language Models to Follow Instructions with Human Feedback*. DOI: 10.48550/arXiv.2203.02155. arXiv: 2203.02155 [cs]. (Visited on 04/27/2026).

Richards, Jack C (2006). *Communicative language teaching today*. Vol. 25. 2. Cambridge university press Cambridge.

Savignon, Sandra J (1987). "Communicative language teaching". *Theory into practice* 26.4, pp. 235–242.

Schmale, Günter (2004). "Kommunikative Kompetenz durch Fremdsprachenunterricht? – Zum Nutzen konversationsanalytischer Erkenntnisse und Verfahren für die Fremdsprachendidaktik". In: *Linguistik für die Fremdsprache Deutsch*. Ed. by Heinz-Helmut Lüger and Rainer Rothenhäusler. Beiträge zur Fremdsprachenvermittlung, Sonderheft 7, pp. 257–281.

Schmale, Günter (2022). "Formulaic Expressions for Foreign Language Learning and Teaching". *Linguistik Online* 113.1, pp. 91–110. ISSN: 1615-3014. DOI: 10.13092/lo.113.8328. (Visited on 04/06/2026).

Shanahan, Murray (2022). *Talking about Large Language Models*. arXiv: 2212.03551 [cs.CL]. URL: https://arxiv.org/abs/2212.03551 (visited on 03/24/2026).

Sinclair, John McHardy (1991). *Corpus, Concordance, Collocation*. (Describing English Language). Oxford: Oxford university press. ISBN: 978-0-19-437144-5.

Spiro, Rand, Paul J. Feltovich, Michael Jacobson, and Richard Coulson (1992). "Cognitive Flexibility, Constructivism, and Hypertext: Random Access Instruction for Advanced Knowledge Acquisition in Ill-Structured Domains". *Constructivism and the technology of instruction: A conversation* 35, pp. 57–75.

Stoeckel, Tim (2019). "An Examination of the New General Service List". *Vocabulary Learning and Instruction* 8.1, pp. 53–61. DOI: 10.7820/vli.v08.1.stoeckel.

Sweller, John (1988). "Cognitive Load During Problem Solving: Effects on Learning". *Cognitive Science* 12.2, pp. 257–285. ISSN: 1551-6709. DOI: 10.1207/s15516709cog1202_4. (Visited on 01/31/2026).

Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin (2017). "Attention Is All You Need". *arXiv preprint arXiv:1706.03762*. DOI: 10.48550/arXiv.1706.03762. (Visited on 04/26/2026).

Zhao, Yize and Christos Thrampoulidis (Oct. 2025). *Geometry of Semantics in Next-Token Prediction: How Optimization Implicitly Organizes Linguistic Representations*. DOI: 10.48550/arXiv.2505.08348. arXiv: 2505.08348 [cs]. (Visited on 04/26/2026).

Zheng, Lianmin, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica (2023). *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. DOI: 10.48550/arXiv.2306.05685. (Visited on 01/31/2026).

Documentation of AI tools used

Table 8: Table documenting AI tools used

	AI Tool	**Application**	**Used in section(s)**	**Comments**
1	Gemini 2.5 Flash	Corpus Generation (SCC)	Synthetic Control Corpus	Via Python API
2	Gemini 2.5 Flash	Semantic Classification	Inter-Rater Reliability Assessment	LLM-as-a-judge
3	Gemini 3 Pro	Ideation & Structure	–	Figures 6–10
4	Claude Sonnet 4.5	Code & Debugging	Appendix E	–
5	Grammarly	Stylistic Proofreading	–	–
6	GitHub Copilot	Repository Privacy Audit	Appendix E	–

AI tools used

1. Gemini 2.5 Flash. Google. *Generate 100 sentences for the verb [lemma]* (Full prompt in Appendix D). 25/02/2026, https://aistudio.google.com.

2. Gemini 2.5 Flash. Google. *Classify the following sentence as LITERAL or IDIOMATIC* (Full prompt in Appendix D). 28/02/2026, https://aistudio.google.com.

3. Gemini 3 Pro. Google. *I am starting my Bachelor's thesis in English Linguistics* (Full documentation in Appendix E). 20/11/2025, https://gemini.google.com.

4. Claude Sonnet 4.5. Anthropic. *Code & Debugging*. 21/11/2025-25/03/2026, https://claude.ai.

5. Grammarly. Grammarly. *Post-drafting spelling and grammatical review.* 28/03/2026, https://grammarly.com.

6. GitHub Copilot. GitHub. *Automated auditing of repository code to ensure data privacy and prevent API key/credential leakage.* 07/02/2026, https://github.com/features/copilot.

Plagiarism Statement

I hereby declare that I have independently written the present work and have not used any aids other than those indicated. Any passages taken verbatim or in substance from other works (including those from electronic databases or the internet) have been clearly marked as such, with proper citation according to academic referencing rules. This declaration also applies to any visual representations, tables, maps, and drawings included in the work. I know any deception will be penalised according to the applicable study and examination regulations.

A Large Language Model (LLM) was used during the ideation process; however, the text and ideas presented are my own unless otherwise specified.

Regensburg, May 5, 2026

A TEC Source Textbooks (B1/B2 Subset)

The following textbook titles (B1/B2-level subset) were included in the TEC-derived NRC sample:

- *Achievers B1* (2015)
- *Achievers B1+* (2015)
- *Achievers B2* (2015)
- *Hi there!* 3e, Niveau A2–B1 (2015)
- *Join the Team* 4e, A2–B1 (2012)
- *Join the Team* 3e, A2–B1 (2013)
- *New Missions* 2de, A2–B1 (2014)
- *Solutions Pre-Intermediate* (2017)
- *Solutions Intermediate* (2016)
- *Solutions Intermediate Plus* (2017)

B SCC Pilot Study Visualisations

The figures in this appendix document exploratory SCC pilot outputs used to refine prompt design and generation parameters before the full main study (see Pilot Study & Prompt Refinement). They are included for transparency and should be read as preliminary diagnostics rather than final inferential results (see Figure 2 and Figure 3).

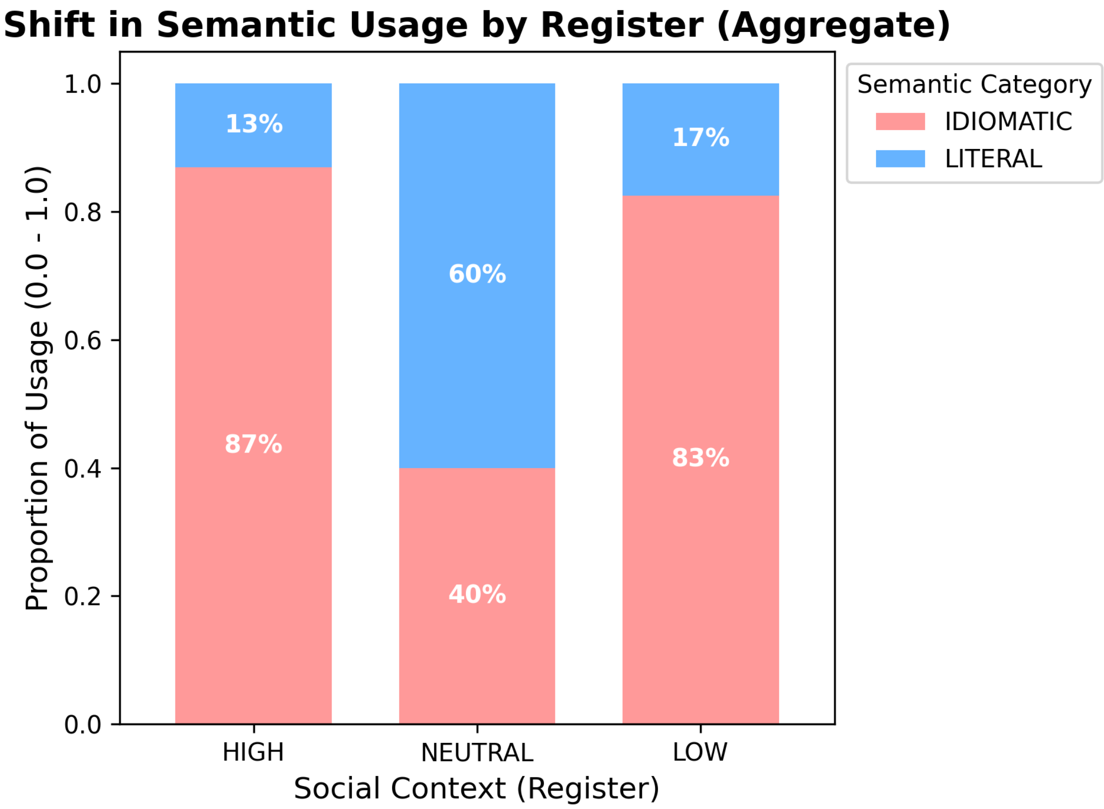


Figure 2: Aggregate proportion of `LITERAL` vs. `IDIOMATIC` usages by register (SCC pilot data).

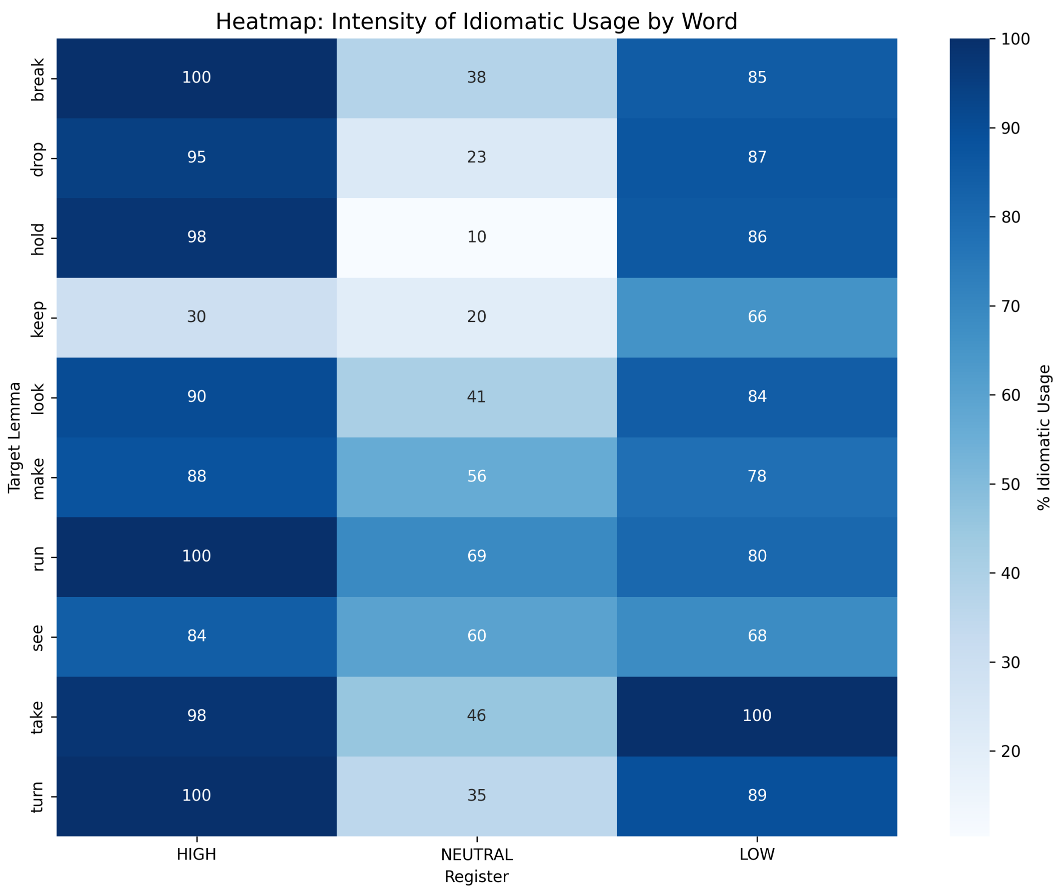


Figure 3: Per-lemma heatmap of `IDIOMATIC` usage rates by register (SCC pilot data).

B.1 Short Interpretation: *run*

In the pilot data, *run* is classified as 100% `IDIOMATIC` in the HIGH register. This aligns with the "Workplace/Professional" prompt condition: in professional contexts, *run* is typically used metaphorically (e.g. *run a meeting*, *run tests*, *run a project*) rather than to describe physical motion. For example, the synthetic HIGH-register sentence "Could you confirm if the new software will run efficiently on older systems?" uses *run* in the sense of "operate/function," i.e. a non-literal (de-lexicalised) usage.

In the LOW register, *run* is 80% `IDIOMATIC`. This is expected because casual contexts plausibly include literal movement (e.g. *I ran to the store*), increasing the share of `LITERAL` uses.

B.2 Short Interpretation: *take*

The word *take* shows a V-shaped pattern. In HIGH (formal register), 98% of uses are `IDIOMATIC`, light-verb constructions like *take responsibility* or *take measures*, where *take* carries minimal semantic weight (e.g. from the SCC: "Might this project take an unexpected turn given the new market analysis?"). In NEUTRAL (instructions), this drops to 46%, with *take* used literally (*Take an exit*, *Take one tablet*). In LOW (casual), idiomaticity rises back to 100%, because of phrasal verbs like *take off* or *take it easy* (e.g. from the SCC: “She always takes her time getting ready, it’s so annoying.”).

In the SCC pilot, both formal and casual registers show high idiomaticity, while the AI-generated instructional register shows substantially lower idiomaticity. This contrast motivated the subsequent NRC comparison, whose full inferential results are reported in Results.

C Final Dataset Visualisations

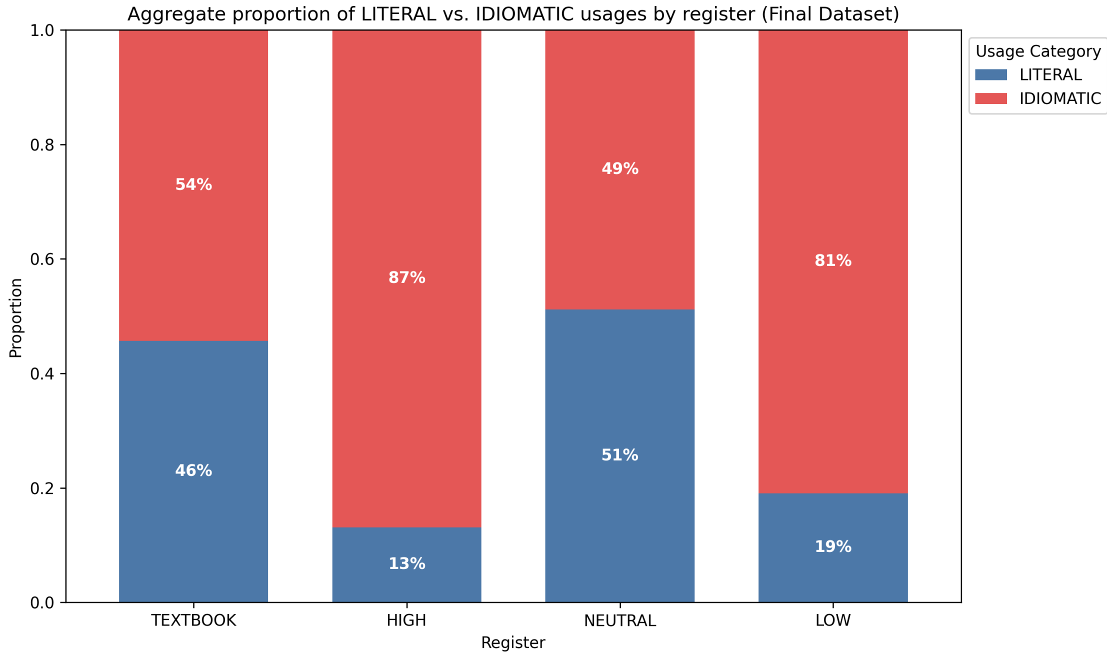
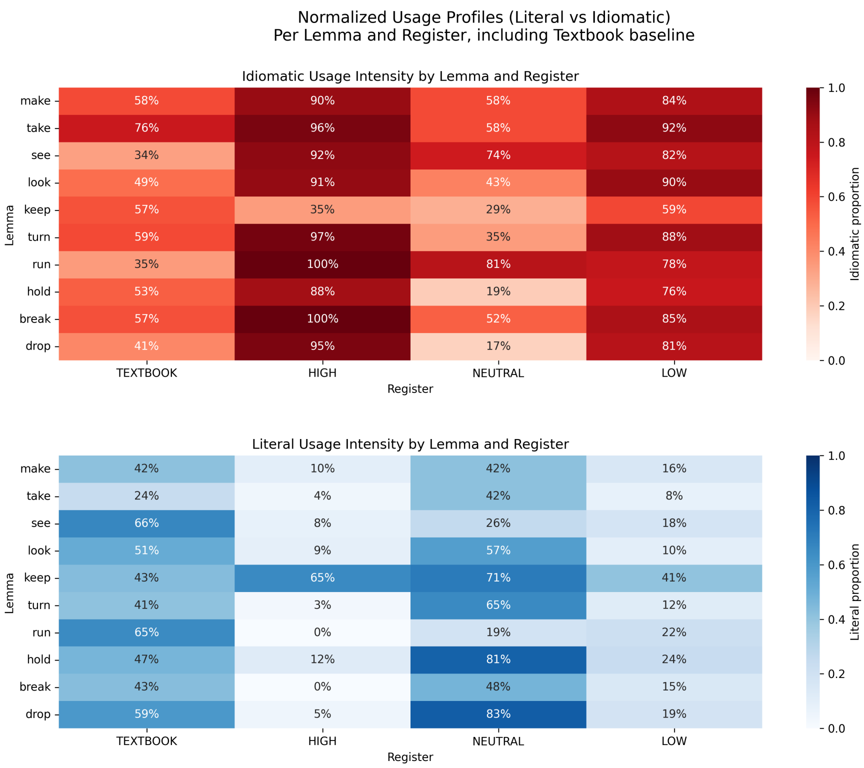


Figure 4: Aggregate proportion of LITERAL vs. IDIOMATIC usages by register (Final Dataset).



Lemma	TEXTBOOK	HIGH	NEUTRAL	LOW
make	58%	90%	58%	84%
take	76%	96%	58%	92%
see	34%	92%	74%	82%
look	49%	91%	43%	90%
keep	57%	35%	29%	59%
turn	59%	97%	35%	88%
run	35%	100%	81%	78%
hold	53%	88%	19%	76%
break	57%	100%	52%	85%
drop	41%	95%	17%	81%

Lemma	TEXTBOOK	HIGH	NEUTRAL	LOW
make	42%	10%	42%	16%
take	24%	4%	42%	8%
see	66%	8%	26%	18%
look	51%	9%	57%	10%
keep	43%	65%	71%	41%
turn	41%	3%	65%	12%
run	65%	0%	19%	22%
hold	47%	12%	81%	24%
break	43%	0%	48%	15%
drop	59%	5%	83%	19%

Figure 5: Per-lemma heatmap of IDIOMATIC usage rates by register (Final Dataset). Note: Cells displaying 100% idiomaticity (e.g., *run* in HIGH) are based on small cell sizes and should be interpreted with caution regarding absolute categorical boundaries.

These figures provide visual representations of the final semantic classification data ($N = 6,720$). Figure 4 shows the aggregate shift in idiomatic usage across prompt conditions, while Figure 5 shows the lemma-specific intensity of these phraseological shifts.

D Python Code Documentation

The following scripts document the computational pipeline. To ensure exact reproducibility, it must be noted that `pos_taggerV2NLTK.py` (Section D.5) was exclusively applied to filter the AI-generated Synthetic Control Corpus (SCC), while `pos_tagger_nltk_for_csv.py` (Section D.6) was engineered specifically to parse the human-authored Natural Reference Corpus (NRC). The code uses simplified file paths for readability. The actual repository implementation uses organised directories (`outputs/`, `data/`, `src/`) and environment variables for API keys. See the GitHub repository for the actual code, dataset and version history.

https://github.com/Eidolom/llm-corpus-generation-analysis

D.1 target_words.txt

```
take
make
hold
keep
break
run
drop
turn
see
look
```

D.2 sketch_engine_converter_nltk.py

```python
import json
import re
import pandas as pd
import nltk
from nltk.tokenize import sent_tokenize, word_tokenize
from nltk.tag import pos_tag

# --- NLTK SETUP ---
# NLTK requires downloading model files (tokenizers, taggers).
try:
    nltk.data.find('tokenizers/punkt_tab')
    nltk.data.find('taggers/averaged_perceptron_tagger_eng')
except LookupError:
    print("Downloading necessary NLTK models...")
```

```python
    nltk.download('punkt_tab')
    nltk.download('averaged_perceptron_tagger_eng')

# --- CONFIGURATION ---
INPUT_FILE = 'textbook_sentences.json'
OUTPUT_JSON = 'semantic_analysis_results.json'
OUTPUT_CSV = 'semantic_analysis_summary.csv'

def clean_text(text):
    """Cleans messy textbook artifacts."""
    if not text: return ""
    # Remove double single quotes often found in SQL dumps
    text = text.replace("''", "'")
    # Remove excessive whitespace
    text = re.sub(r'\s+', ' ', text).strip()
    return text

def get_context_window(words, index, window_size=5):
    """
    Since NLTK doesn't do full dependency parsing easily,
    we grab a 'context window' of words following the verb
    to capture the object/phrase (e.g., 'keep' -> 'in touch').
    """
    start = index + 1
    end = min(index + 1 + window_size, len(words))
    context_words = words[start:end]
    return " ".join(context_words)

def analyze_data(data):
    analyzed_rows = []
    print(f"Processing {len(data)} raw entries with NLTK...")

    for entry in data:
        raw_text = clean_text(entry.get('sentence', ''))
        target_lemma = entry.get('lemma', '').lower()

        # 1. Split chunk into sentences
        sentences = sent_tokenize(raw_text)

        for sent in sentences:
            # 2. Tokenize words
            words = word_tokenize(sent)

            # 3. Part-of-Speech Tagging
```

```
            # Returns list of tuples: [('Word', 'TAG'), ...]
            tagged_words = pos_tag(words)

            found_verb = False
            context_pattern = "N/A"

            for i, (word, tag) in enumerate(tagged_words):
                # Check if word matches target (normalized)
                # AND if tag starts with 'V' (VB, VBD, VBG, VBN, VBP, VBZ are
    all verbs)
                if word.lower() == target_lemma and tag.startswith('V'):
                    found_verb = True
                    # Grab the words following the verb
                    context_pattern = get_context_window(words, i)
                    break

            # 4. Filter and Save
            # We filter out very short fragments (e.g. headers)
            if found_verb and len(words) > 3:
                row = {
                    "Target_Lemma": target_lemma,
                    "Extracted_Sentence": sent,
                    "Context_Pattern": f"{target_lemma} + {context_pattern}",
                    "Original_Source": entry.get('Source', 'Unknown'),
                    "Register": entry.get('register', 'N/A')
                }
                analyzed_rows.append(row)

    return analyzed_rows

# --- MAIN EXECUTION ---
if __name__ == "__main__":
    print("--- STARTING SEMANTIC ANALYSIS (NLTK VERSION) ---")

    # 1. Load Data
    try:
        with open(INPUT_FILE, 'r', encoding='utf-8') as f:
            raw_data = json.load(f)
    except FileNotFoundError:
        print(f"Error: Could not find '{INPUT_FILE}'. Make sure it exists.")
        exit()

    # 2. Analyze
    results = analyze_data(raw_data)
```

```
    # 3. Save Results
    if results:
        with open(OUTPUT_JSON, 'w', encoding='utf-8') as f:
            json.dump(results, f, indent=4, ensure_ascii=False)

        df = pd.DataFrame(results)
        df.to_csv(OUTPUT_CSV, index=False)

        print(f"   Analysis Complete.")
        print(f"   Processed {len(raw_data)} chunks into {len(results)}
    specific sentences.")
        print(f"   Saved to '{OUTPUT_CSV}'.")
    else:
        print("No valid sentences containing the target verbs were found.")
```

D.3 debug.py

```
import google.generativeai as genai

# Paste key directly here for the test
API_KEY = "API_KEY_HERE"

genai.configure(api_key=API_KEY)
model = genai.GenerativeModel('gemini-2.5-flash')

print("--- TESTING GEMINI API ---")

try:
    response = model.generate_content("Write a sentence using the word 'run'.")
    print(f"Success! Response: {response.text}")
except Exception as e:
    print(f"Error: {e}")
```

D.4 scalable_context_generator.py

```
import google.generativeai as genai
import time
import json
import re
from pathlib import Path

# --- CONFIGURATION ---
```

```
API_KEY = "API_KEY_HERE"
TARGET_WORDS_FILE = "data/target_words.txt"
OUTPUT_FILENAME = "outputs/intermediate_sentences.json"

# --- GENERATION SETTINGS ---
# 5 Batches x 10 sentences = 50 sentences per register (150 total per word)
NUM_BATCHES = 5
SENTENCES_PER_REG_PER_BATCH = 10

genai.configure(api_key=API_KEY)
model = genai.GenerativeModel('gemini-2.5-flash') # 2.5-flash is best for high-
    volume tasks

SYSTEM_PROMPT = """
You are a linguistic expert specialising in pragmatics and corpus generation.
Your task is to generate distinct, high-quality sentences for a target word
    based on strict register constraints.
The sentences must be appropriate for a CEFR B1/B2 learner.
"""

def clean_and_load_json(raw_text):
    """Clean markdown and parse JSON."""
    if not raw_text: return None
    try:
        # Regex to find the first [ and the last ]
        match = re.search(r'\[.*\]', raw_text, re.DOTALL)
        if match:
            return json.loads(match.group(0))
        else:
            # Fallback for simple strip
            cleaned = raw_text.replace('```json', '').replace('```', '').strip
    ()
            return json.loads(cleaned)
    except Exception as e:
        print(f"   JSON Parse Error: {e}")
        return None

def load_target_words(filename):
    try:
        with open(filename, 'r', encoding='utf-8') as f:
            return [line.strip() for line in f if line.strip()]
    except FileNotFoundError:
        print(f" Error: '{filename}' not found.")
        return []
```

```
def generate_batch(word, batch_num):
    """Generates a single batch of 30 sentences (10 High, 10 Low, 10 Neutral).
    """

    prompt = f"""
    Target Word: "{word}"
    Batch ID: {batch_num} (Ensure these sentences are unique from generic
    examples)

    Generate a JSON array containing exactly {3 * SENTENCES_PER_REG_PER_BATCH}
    sentences:

    1. {SENTENCES_PER_REG_PER_BATCH} sentences: Register HIGH (Formal), Mood
    Question. Context: Workplace/Professional.
    2. {SENTENCES_PER_REG_PER_BATCH} sentences: Register LOW (Casual), Mood
    Statement. Context: Friends/Personal/Slang.
    3. {SENTENCES_PER_REG_PER_BATCH} sentences: Register NEUTRAL (Direct), Mood
     Imperative. Context: Instructions/Navigation/Rules.

    Vary the sentence structure and vocabulary usage.
    Output JSON keys: "register", "mood", "sentence".
    """

    try:
        response = model.generate_content(
            contents=[{"role": "user", "parts": [{"text": SYSTEM_PROMPT + "\n"
    + prompt}]}]
        )
        return clean_and_load_json(response.text)
    except Exception as e:
        print(f"   API Error: {e}")
        return None

def main():
    words = load_target_words(TARGET_WORDS_FILE)
    if not words: return

    all_data = []
    total_words = len(words)

    print(f"--- STARTING GENERATION: {total_words} Words x {NUM_BATCHES}
    Batches ---")
```

```
print(f"Target: {NUM_BATCHES * SENTENCES_PER_REG_PER_BATCH} sentences per register per word.")

for i, word in enumerate(words):
    word_sentences = []
    print(f"\n[{i+1}/{total_words}]  Processing ’{word}’...")

    for batch_idx in range(NUM_BATCHES):
        print(f"   -> Generating Batch {batch_idx+1}/{NUM_BATCHES}...", end="", flush=True)

        batch_data = generate_batch(word, batch_idx)

        if batch_data:
            # Validate we actually got a list
            if isinstance(batch_data, list):
                # Add metadata
                for item in batch_data:
                    item[’lemma’] = word
                    item[’Source’] = "Synthetic_V2"
                    item[’CEFR_Target’] = "B1/B2"
                    # Normalize register names just in case
                    if ’register’ in item:
                        item[’register’] = item[’register’].upper()

                word_sentences.extend(batch_data)
                print(f"  Got {len(batch_data)} sentences.")
            else:
                print(f"  format error (not a list).")
        else:
            print("  Failed.")

        # Sleep slightly longer to avoid rate limits on the loop
        time.sleep(2.0)

    all_data.extend(word_sentences)
    print(f"    Total for ’{word}’: {len(word_sentences)} sentences.")

# Save
if all_data:
    Path(OUTPUT_FILENAME).parent.mkdir(parents=True, exist_ok=True)
    with open(OUTPUT_FILENAME, ’w’, encoding=’utf-8’) as f:
        json.dump(all_data, f, indent=4)
    print(f"\nSUCCESS! Generated {len(all_data)} total sentences.")
```

```
        print(f"Saved to {OUTPUT_FILENAME}")
    else:
        print("\n No data generated.")

if __name__ == "__main__":
    main()
```

D.5 pos_taggerV2NLTK.py

```
import sys
import json
import pandas as pd
from pathlib import Path
import nltk
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer

project_root = Path(__file__).parent.parent.parent
if str(project_root) not in sys.path:
    sys.path.insert(0, str(project_root))

INPUT_FILENAME = "outputs/intermediate_sentences.json"
OUTPUT_FILENAME = "outputs/thesis_pragmatic_data_filtered.csv"

nltk.download('punkt', quiet=True)
nltk.download('averaged_perceptron_tagger', quiet=True)
nltk.download('wordnet', quiet=True)

lemmatizer = WordNetLemmatizer()

def load_sentences(filename):
    """Loads sentences from the intermediate JSON file."""
    try:
        with open(filename, 'r', encoding='utf-8') as f:
            return json.load(f)
    except Exception as e:
        print(f"Error loading file: {e}")
        return None

def main():
    print("--- STARTING NLTK POS-GATEKEEPER ---")
    all_sentences = load_sentences(INPUT_FILENAME)
    if not all_sentences:
        return
```

```python
    print(f"Total raw sentences loaded: {len(all_sentences)}")

    valid_sentences = []
    dropped_count = 0

    for item in all_sentences:
        target_lemma = item['lemma'].lower()
        sentence_text = item['sentence']

        tokens = word_tokenize(sentence_text)
        pos_tags = nltk.pos_tag(tokens)

        is_valid_verb = False

        for token, tag in pos_tags:
            if tag.startswith('V'):
                token_lemma = lemmatizer.lemmatize(token.lower(), pos='v')

                if token_lemma == target_lemma:
                    is_valid_verb = True
                    break

        if is_valid_verb:
            valid_sentences.append(item)
        else:
            dropped_count += 1

    print(f"\nFiltering Complete!")
    print(f"Sentences Kept: {len(valid_sentences)}")
    print(f"Sentences Dropped (Nouns/Adjectives): {dropped_count}")

    if valid_sentences:
        df = pd.DataFrame(valid_sentences)
        df.to_csv(OUTPUT_FILENAME, index=False)
        print(f"Clean, verb-only dataset saved to '{OUTPUT_FILENAME}'.")

    else:
        print("\nFailed to generate the final CSV. No valid verbs found.")

if __name__ == "__main__":
    main()
```

D.6 pos_tagger_nltk_for_csv.py

```
import pandas as pd
import nltk
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer

# --- CONFIGURATION ---
INPUT_FILENAME = "outputs/thesis_semantic_data_final.csv" # Update path if
    needed
OUTPUT_FILENAME = "outputs/thesis_textbook_data_filtered.csv"      # Update
    path if needed

# Download required NLTK datasets (quietly)
nltk.download('punkt', quiet=True)
nltk.download('averaged_perceptron_tagger', quiet=True)
nltk.download('wordnet', quiet=True)

lemmatizer = WordNetLemmatizer()

def main():
    print("--- STARTING NLTK POS-GATEKEEPER (CSV VERSION) ---")

    try:
        # Load the CSV
        df = pd.read_csv(INPUT_FILENAME)
    except Exception as e:
        print(f"Error loading CSV: {e}")
        return

    print(f"Total raw sentences loaded: {len(df)}")

    valid_indices = []

    # Identify the correct column names (handling uppercase/lowercase
    differences)
    text_col = 'Full_Sentence' if 'Full_Sentence' in df.columns else 'sentence'
    lemma_col = 'Lemma' if 'Lemma' in df.columns else 'lemma'

    # 1. The Lemmatize-Before-Filtering Logic
    for index, row in df.iterrows():
        target_lemma = str(row[lemma_col]).lower().strip()
        sentence_text = str(row[text_col])

        # Tokenize and tag the sentence
```

```
        tokens = word_tokenize(sentence_text)
        pos_tags = nltk.pos_tag(tokens)

        is_valid_verb = False

        # Scan the sentence for our target verb
        for token, tag in pos_tags:
            # Check if NLTK tagged it as ANY type of verb (VB, VBD, VBG, VBN,
    VBP, VBZ)
            if tag.startswith('V'):
                # Lemmatise the verb
                token_lemma = lemmatizer.lemmatize(token.lower(), pos='v')

                # Check if it matches our target lemma
                if token_lemma == target_lemma:
                    is_valid_verb = True
                    break # Found it acting as a verb, keep the row

        if is_valid_verb:
            valid_indices.append(index)

    # 2. Filter the dataframe to only keep valid rows
    clean_df = df.loc[valid_indices]
    dropped_count = len(df) - len(clean_df)

    print(f"\nFiltering Complete!")
    print(f"Sentences Kept: {len(clean_df)}")
    print(f"Sentences Dropped (Nouns/Adjectives): {dropped_count}")

    # 3. Export the clean data
    if not clean_df.empty:
        from pathlib import Path
        Path(OUTPUT_FILENAME).parent.mkdir(parents=True, exist_ok=True)
        clean_df.to_csv(OUTPUT_FILENAME, index=False)
        print(f"Clean, verb-only dataset saved to '{OUTPUT_FILENAME}'.")
    else:
        print("\nFailed to generate the final CSV. No valid verbs found.")

if __name__ == "__main__":
    main()
```

D.7 semantic_analyzer.py

```
import google.generativeai as genai
```

```python
import pandas as pd
import time
import json
import re
import math
from pathlib import Path

# --- CONFIGURATION ---
API_KEY = "API_KEY_HERE"
INPUT_FILENAME = "outputs/intermediate_sentences.json"
OUTPUT_FILENAME = "outputs/thesis_semantic_data_final.csv"

# BATCH SIZE FOR ANALYSIS
# We will process 20 sentences at a time. This is the "Safe Zone" for LLM
    counting.
CHUNK_SIZE = 20

genai.configure(api_key=API_KEY)
model = genai.GenerativeModel('gemini-2.5-flash')

SYSTEM_PROMPT = """
You are a semantic analyzer. I will provide a list of sentences containing a
    target word.
Classify the usage of the target word in each sentence into exactly one
    category:
1. "LITERAL" (Physical/Core meaning).
2. "IDIOMATIC" (De-lexicalized, Metaphor, Phrasal Verb, or Fixed Phrase).

Return a strict JSON ARRAY of strings.
Example Output: ["LITERAL", "IDIOMATIC", "LITERAL"]
"""

def load_sentences(filename):
    try:
        with open(filename, 'r', encoding='utf-8') as f:
            return json.load(f)
    except Exception as e:
        print(f" Error loading input file: {e}")
        return None

def get_tags_for_chunk(word, chunk_sentences, retry_count=0):
    """
    Analyzes a small chunk of sentences.
    """
```

```
prompt = f"""
Target Word: "{word}"
Sentences to classify ({len(chunk_sentences)} items):
{json.dumps(chunk_sentences)}

Return exactly {len(chunk_sentences)} tags in a JSON array.
"""

try:
    response = model.generate_content(
        contents=[{"role": "user", "parts": [{"text": SYSTEM_PROMPT + "\n"
+ prompt}]}]
    )

    raw_text = response.text.strip()

    # Cleaner Regex to find JSON array
    match = re.search(r'\[.*?\]', raw_text, re.DOTALL)
    if not match:
        # If regex fails, try parsing raw text if it looks like a list
        if raw_text.startswith('[') and raw_text.endswith(']'):
            json_str = raw_text
        else:
            raise ValueError("No JSON array found")
    else:
        json_str = match.group(0)

    tags = json.loads(json_str)

    # Validate Count
    if len(tags) != len(chunk_sentences):
        # Retry once if count is wrong
        if retry_count < 1:
            print(f"     Count mismatch ({len(tags)} vs {len(
chunk_sentences)}). Retrying...")
            time.sleep(1)
            return get_tags_for_chunk(word, chunk_sentences, retry_count=1)
        else:
            print(f"     Failed chunk for '{word}'. Expected {len(
chunk_sentences)}, got {len(tags)}.")
            return ["ERROR"] * len(chunk_sentences)

    return [str(t).upper() for t in tags]
```

```
    except Exception as e:
        print(f"     Error: {e}")
        return ["ERROR"] * len(chunk_sentences)

def main():
    data = load_sentences(INPUT_FILENAME)
    if not data: return

    # Group by Lemma
    grouped_data = {}
    for item in data:
        lemma = item['lemma']
        if lemma not in grouped_data: grouped_data[lemma] = []
        grouped_data[lemma].append(item)

    final_rows = []
    sorted_words = sorted(grouped_data.keys())
    total_words = len(sorted_words)

    print(f"--- STARTING CHUNKED ANALYSIS FOR {total_words} WORDS ---")

    for i, word in enumerate(sorted_words):
        records = grouped_data[word]
        total_items = len(records)
        print(f"[{i+1}/{total_words}]  Analyzing '{word}' ({total_items}
    sentences)...")

        # --- THE CHUNKING LOOP ---
        word_tags = []
        num_chunks = math.ceil(total_items / CHUNK_SIZE)

        for batch_idx in range(num_chunks):
            start = batch_idx * CHUNK_SIZE
            end = start + CHUNK_SIZE
            chunk_records = records[start:end]
            chunk_sentences = [r['sentence'] for r in chunk_records]

            # Get tags for just this small batch
            tags = get_tags_for_chunk(word, chunk_sentences)
            word_tags.extend(tags)

            # print(f"    Batch {batch_idx+1}/{num_chunks}: Processed {len(tags
    )} items.")
            time.sleep(0.5) # Fast sleep for small chunks
```

```
        # Merge tags back
        for record, tag in zip(records, word_tags):
            row = {
                "Lemma": record['lemma'],
                "Register": record['register'],
                "Mood": record['mood'],
                "Usage_Category": tag,
                "Full_Sentence": record['sentence']
            }
            final_rows.append(row)

    # Save
    df = pd.DataFrame(final_rows)
    Path(OUTPUT_FILENAME).parent.mkdir(parents=True, exist_ok=True)
    df.to_csv(OUTPUT_FILENAME, index=False)

    print("\n" + "="*40)
    print(f" DONE. Saved to {OUTPUT_FILENAME}")
    print("Summary of Results:")
    print(df['Usage_Category'].value_counts())
    print("="*40)

if __name__ == "__main__":
    main()
```

D.8 irr_sheet_generator.py

```
"""
Generate blinded IRR annotation material for the thesis workflow.

Outputs:
1) Files/irr_annotation_sheet.csv
   - Sheet used for manual coding (no model labels or register metadata).
2) Files/irr_gold_key.csv
   - Hidden key containing model labels for later agreement calculation.

Design:
- 100 NRC items (TEXTBOOK)
- 100 SCC items (stratified across HIGH / NEUTRAL / LOW)
- Deterministic random seed for reproducibility
"""

import pandas as pd
```

```
# --- CONFIGURATION ---
NRC_INPUT_FILE = "Files/thesis_semantic_data_final.csv"
SCC_INPUT_FILE = "Files/thesis_semantic_data_final_2.csv"

ANNOTATION_OUTPUT_FILE = "Files/irr_annotation_sheet.csv"
KEY_OUTPUT_FILE = "Files/irr_gold_key.csv"

NRC_SAMPLE_SIZE = 100
SCC_SAMPLE_SIZE = 100
RANDOM_SEED = 42

REQUIRED_COLUMNS = ["Lemma", "Register", "Mood", "Usage_Category", "
    Full_Sentence"]
SCC_REGISTER_ORDER = ["HIGH", "NEUTRAL", "LOW"]


def validate_columns(df: pd.DataFrame, source_name: str) -> None:
    missing = [column for column in REQUIRED_COLUMNS if column not in df.
    columns]
    if missing:
        raise ValueError(f"{source_name} is missing required columns: {missing}
    ")


def sample_nrc(nrc_df: pd.DataFrame, n: int, seed: int) -> pd.DataFrame:
    nrc_pool = nrc_df[nrc_df["Register"].astype(str).str.upper() == "TEXTBOOK"
    ].copy()
    if len(nrc_pool) < n:
        raise ValueError(f"NRC pool too small: requested {n}, available {len(
    nrc_pool)}")

    sample = nrc_pool.sample(n=n, random_state=seed).copy()
    sample["Source_Group"] = "NRC"
    return sample


def sample_scc_stratified(scc_df: pd.DataFrame, n_total: int, seed: int) -> pd.
    DataFrame:
    scc_df = scc_df.copy()
    scc_df["Register"] = scc_df["Register"].astype(str).str.upper()
```

```
    missing_registers = [reg for reg in SCC_REGISTER_ORDER if reg not in set(
    scc_df["Register"])]
    if missing_registers:
        raise ValueError(f"SCC data is missing required registers: {
    missing_registers}")

    base = n_total // len(SCC_REGISTER_ORDER)
    remainder = n_total % len(SCC_REGISTER_ORDER)

    allocations = {
        register: base + (1 if idx < remainder else 0)
        for idx, register in enumerate(SCC_REGISTER_ORDER)
    }

    sampled_chunks = []
    for idx, register in enumerate(SCC_REGISTER_ORDER):
        n_register = allocations[register]
        register_pool = scc_df[scc_df["Register"] == register]

        if len(register_pool) < n_register:
            raise ValueError(
                f"SCC pool for {register} too small: requested {n_register},
    available {len(register_pool)}"
            )

        sampled_chunk = register_pool.sample(n=n_register, random_state=seed +
    idx).copy()
        sampled_chunks.append(sampled_chunk)

    sample = pd.concat(sampled_chunks, ignore_index=True)
    sample["Source_Group"] = "SCC"
    return sample


def build_outputs(nrc_sample: pd.DataFrame, scc_sample: pd.DataFrame, seed: int
    ) -> tuple[pd.DataFrame, pd.DataFrame]:
    combined = pd.concat([nrc_sample, scc_sample], ignore_index=True)
    combined = combined.sample(frac=1.0, random_state=seed).reset_index(drop=
    True)

    combined["IRR_ID"] = [f"IRR_{idx:04d}" for idx in range(1, len(combined) +
    1)]
    combined["Model_Label"] = combined["Usage_Category"].astype(str).str.upper
    ()
```

```
    annotation_sheet = combined[["IRR_ID", "Lemma", "Full_Sentence"]].rename(
        columns={
            "IRR_ID": "irr_id",
            "Lemma": "lemma",
            "Full_Sentence": "sentence",
        }
    )
    annotation_sheet["human_label"] = ""
    annotation_sheet["notes"] = ""

    gold_key = combined[
        [
            "IRR_ID",
            "Source_Group",
            "Register",
            "Mood",
            "Lemma",
            "Model_Label",
            "Full_Sentence",
        ]
    ].rename(
        columns={
            "IRR_ID": "irr_id",
            "Source_Group": "source_group",
            "Register": "register",
            "Mood": "mood",
            "Lemma": "lemma",
            "Model_Label": "model_label",
            "Full_Sentence": "sentence",
        }
    )

    return annotation_sheet, gold_key


def main() -> None:
    print("--- Generating IRR annotation sheet ---")

    nrc_df = pd.read_csv(NRC_INPUT_FILE)
    scc_df = pd.read_csv(SCC_INPUT_FILE)

    validate_columns(nrc_df, "NRC input")
    validate_columns(scc_df, "SCC input")
```

```python
    nrc_sample = sample_nrc(nrc_df, n=NRC_SAMPLE_SIZE, seed=RANDOM_SEED)
    scc_sample = sample_scc_stratified(scc_df, n_total=SCC_SAMPLE_SIZE, seed=
    RANDOM_SEED)

    annotation_sheet, gold_key = build_outputs(nrc_sample, scc_sample, seed=
    RANDOM_SEED)

    annotation_sheet.to_csv(ANNOTATION_OUTPUT_FILE, index=False)
    gold_key.to_csv(KEY_OUTPUT_FILE, index=False)

    print(f"Saved annotation sheet: {ANNOTATION_OUTPUT_FILE}")
    print(f"Saved gold key: {KEY_OUTPUT_FILE}")
    print(f"Total IRR sample size: {len(annotation_sheet)}")

    source_counts = gold_key["source_group"].value_counts().to_dict()
    register_counts = gold_key["register"].value_counts().to_dict()
    print(f"Source balance: {source_counts}")
    print(f"Register balance: {register_counts}")


if __name__ == "__main__":
    main()
```

D.9 compute_split_irr.py

```python
"""
Split IRR analysis for NRC vs SCC.

This script compares human labels to model labels and reports:
- aggregate Cohen's kappa
- NRC-only Cohen's kappa
- SCC-only Cohen's kappa

Expected inputs
---------------
- Files/irr_annotation_sheet - irr_annotation_sheet.csv.csv
  (human labels)
- Files/irr_master_key.csv (optional)
  (model labels + metadata)

If a master-key file is unavailable, the script can read a merged sheet
that already contains both human and model labels.
"""
```

```python
from __future__ import annotations

from pathlib import Path
from dataclasses import dataclass
import re

import pandas as pd
from sklearn.metrics import cohen_kappa_score


VALID_LABELS = {"LITERAL", "IDIOMATIC"}


@dataclass
class SplitResult:
    group: str
    n_valid: int
    raw_agreement: float | None
    kappa: float | None


def normalize_text(value: object) -> str:
    text = "" if value is None else str(value)
    text = re.sub(r"\s+", " ", text)
    return text.strip().lower()


def normalize_label(value: object) -> str:
    label = "" if value is None else str(value)
    return label.strip().upper()


def first_existing(columns: list[str], candidates: list[str]) -> str | None:
    lower_map = {c.lower(): c for c in columns}
    for candidate in candidates:
        key = candidate.lower()
        if key in lower_map:
            return lower_map[key]
    return None


def compute_group(df: pd.DataFrame, group_name: str) -> SplitResult:
    clean = df.copy()
```

```python
    clean["human_label"] = clean["human_label"].map(normalize_label)
    clean["model_label"] = clean["model_label"].map(normalize_label)

    keep = (
        clean["human_label"].isin(VALID_LABELS)
        & clean["model_label"].isin(VALID_LABELS)
    )
    clean = clean[keep]

    if clean.empty:
        return SplitResult(group_name, 0, None, None)

    raw = (clean["human_label"] == clean["model_label"]).mean()
    kappa = cohen_kappa_score(clean["human_label"], clean["model_label"])
    return SplitResult(group_name, int(len(clean)), float(raw), float(kappa))


def kappa_interpretation(value: float | None) -> str:
    if value is None:
        return "n/a"
    if value < 0.00:
        return "poor"
    if value < 0.20:
        return "slight"
    if value < 0.40:
        return "fair"
    if value < 0.60:
        return "moderate"
    if value < 0.80:
        return "substantial"
    return "almost perfect"


def load_input_frames(project_root: Path) -> pd.DataFrame:
    files_dir = project_root / "Files"

    annotation_path = (
        files_dir / "irr_annotation_sheet - irr_annotation_sheet.csv.csv"
    )
    master_key_path = files_dir / "irr_master_key.csv"

    if not annotation_path.exists():
        raise FileNotFoundError(
            f"Missing annotation file: {annotation_path}"
```

```
    )

ann = pd.read_csv(annotation_path)

human_col = first_existing(
    ann.columns.tolist(),
    ["Human_Label", "human_label", "human", "label_human"],
)
sent_col = first_existing(
    ann.columns.tolist(),
    ["Full_Sentence", "full_sentence", "sentence", "text"],
)

if human_col is None:
    raise ValueError("Could not find human label column.")

ann = ann.rename(columns={human_col: "human_label"})

if master_key_path.exists() and sent_col is not None:
    key = pd.read_csv(master_key_path)
    key_sent = first_existing(
        key.columns.tolist(),
        ["Full_Sentence", "full_sentence", "sentence", "text"],
    )
    key_model = first_existing(
        key.columns.tolist(),
        ["model_label", "Model_Label", "ai_label", "label_model"],
    )
    key_group = first_existing(
        key.columns.tolist(),
        ["source_group", "Source_Group", "group", "corpus"],
    )

    if key_sent and key_model and key_group and sent_col:
        ann["_sent_key"] = ann[sent_col].map(normalize_text)
        key["_sent_key"] = key[key_sent].map(normalize_text)
        keep_cols = ["_sent_key", key_model, key_group]
        key = key[keep_cols].drop_duplicates("_sent_key")
        key = key.rename(
            columns={
                key_model: "model_label",
                key_group: "source_group",
            }
        )
```

```
            merged = ann.merge(key, on="_sent_key", how="left")
            return merged

    model_col = first_existing(
        ann.columns.tolist(),
        ["Model_Label", "model_label", "ai_label", "label_model"],
    )
    group_col = first_existing(
        ann.columns.tolist(),
        ["Source_Group", "source_group", "group", "corpus"],
    )

    if model_col is None or group_col is None:
        raise ValueError(
            "Need either Files/irr_master_key.csv or columns "
            "for model labels and source group in annotation sheet."
        )

    ann = ann.rename(columns={model_col: "model_label", group_col: "
    source_group"})
    return ann


def main() -> None:
    project_root = Path(__file__).resolve().parents[1]
    df = load_input_frames(project_root)

    if "source_group" not in df.columns:
        raise ValueError("Missing source_group column.")
    if "model_label" not in df.columns:
        raise ValueError("Missing model_label column.")

    df["source_group"] = df["source_group"].astype(str).str.upper().str.strip()

    aggregate = compute_group(df, "ALL")
    nrc = compute_group(df[df["source_group"] == "NRC"], "NRC")
    scc = compute_group(df[df["source_group"] == "SCC"], "SCC")

    rows = [aggregate, nrc, scc]

    print("=" * 64)
    print(f"{'Group':<10}{'n':>8}{'Raw Agr.':>12}{'Kappa':>10}  Interpretation"
    )
    print("-" * 64)
```

```python
    for r in rows:
        if r.kappa is None:
            print(f"{r.group:<10}{'-':>8}{'-':>12}{'-':>10}  n/a")
            continue
        raw_pct = f"{r.raw_agreement * 100:.1f}%"
        print(
            f"{r.group:<10}{r.n_valid:>8}{raw_pct:>12}"
            f"{r.kappa:>10.3f}  {kappa_interpretation(r.kappa)}"
        )
    print("=" * 64)

    out_dir = project_root / "outputs"
    out_dir.mkdir(exist_ok=True)
    out_path = out_dir / "irr_split_results.csv"

    out_df = pd.DataFrame(
        {
            "group": [r.group for r in rows],
            "n_valid": [r.n_valid for r in rows],
            "raw_agreement": [r.raw_agreement for r in rows],
            "kappa": [r.kappa for r in rows],
            "interpretation": [kappa_interpretation(r.kappa) for r in rows],
        }
    )
    out_df.to_csv(out_path, index=False)
    print(f"Saved: {out_path}")


if __name__ == "__main__":
    main()
```

D.10 irr_split_results.csv

```
group,n_valid,n_excluded,raw_agreement,kappa
ALL (aggregate),186,14,0.8118,0.6158
NRC (Textbook),87,13,0.7701,0.532
SCC (AI-generated),99,1,0.8485,0.6379
```

D.11 visualizer.py

```python
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from itertools import combinations
from itertools import combinations
from scipy.stats import chi2_contingency, fisher_exact, fisher_exact

# --- CONFIGURATION ---
INPUT_FILENAME = "thesis_semantic_data_final.csv"

# 1. Load Data
df = pd.read_csv(INPUT_FILENAME)

# Clean up Register names just in case (ensure UPPERCASE)
df['Register'] = df['Register'].str.upper()

# Define specific order for charts (Logical progression)
register_order = ['HIGH', 'NEUTRAL', 'LOW']
usage_order = ['LITERAL', 'IDIOMATIC']

# --- PART 1: THE MAIN FINDING (Stacked Bar Chart) ---
print("\n--- 1. GENERATING AGGREGATE CHART ---")

# Calculate counts and percentages
cross_tab = pd.crosstab(df['Register'], df['Usage_Category'])
cross_tab = cross_tab.reindex(index=register_order, columns=usage_order,
    fill_value=0)
cross_tab = cross_tab.loc[cross_tab.sum(axis=1) > 0]
cross_tab_prop = cross_tab.div(cross_tab.sum(1), axis=0)

# Plotting
plt.figure(figsize=(10, 6))
# Plot the bars (stacked)
ax = cross_tab_prop.plot(kind='bar', stacked=True, color=['#ff9999', '#66b3ff'
    ], width=0.7)

# Formatting
plt.title('Shift in Semantic Usage by Register (Aggregate)', fontsize=14,
    fontweight='bold')
plt.ylabel('Proportion of Usage (0.0 - 1.0)', fontsize=12)
plt.xlabel('Social Context (Register)', fontsize=12)
plt.xticks(rotation=0)
plt.legend(title='Semantic Category', loc='upper left', bbox_to_anchor=(1, 1))

# Add percentage labels on the bars
for c in ax.containers:
```

```
    labels = [f"{value * 100:.0f}%" if value > 0 else "" for value in c.
    datavalues]
    ax.bar_label(c, labels=labels, label_type='center', color='white',
    fontweight='bold')

plt.tight_layout()
plt.savefig('results_aggregate_chart.png', dpi=300)
print(" Saved 'results_aggregate_chart.png'")

# --- PART 2: THE STATISTICAL TEST ---
print("\n--- 2. RUNNING CHI-SQUARE TEST ---")
# We test if Register and Usage are independent
chi2, p, dof, expected = chi2_contingency(cross_tab)

print(f"Chi-Square Statistic: {chi2:.4f}")
print(f"P-Value: {p:.4e}")
if p < 0.05:
    print(">> RESULT: Statistically Significant! (You can reject the Null
    Hypothesis)")
else:
    print(">> RESULT: Not Significant.")

# Follow-up analysis for sparse expected frequencies
if (expected < 5).any():
    print("\nExpected frequencies below 5 detected. Running pairwise 2x2 Fisher
     follow-up tests.")
    pairwise_results = []
    for reg_a, reg_b in combinations(cross_tab.index, 2):
        pair_table = cross_tab.loc[[reg_a, reg_b], usage_order]
        if (pair_table.sum(axis=1) == 0).any() or (pair_table.sum(axis=0) == 0)
    .any():
            continue
        odds_ratio, fisher_p = fisher_exact(pair_table.values)
        pairwise_results.append((reg_a, reg_b, odds_ratio, fisher_p))

    if pairwise_results:
        corrected_alpha = 0.05 / len(pairwise_results)
        print(f"Bonferroni-corrected alpha: {corrected_alpha:.4f}")
        for reg_a, reg_b, odds_ratio, fisher_p in pairwise_results:
            significance = "significant" if fisher_p < corrected_alpha else "
    not significant"
            print(
                f"Fisher {reg_a} vs {reg_b}: p={fisher_p:.4e}, "
                f"odds_ratio={odds_ratio:.4f} ({significance})"
```

```
            )
    else:
        print("Pairwise Fisher follow-up skipped (insufficient variation in 2x2
     tables).")

# --- PART 3: THE WORD BREAKDOWN (Heatmap) ---
print("\n--- 3. GENERATING WORD HEATMAP ---")

# Calculate % Idiomatic for every word in every register
pivot = df.groupby(['Lemma', 'Register'])['Usage_Category'].value_counts(
    normalize=True).unstack(fill_value=0)

# We only care about the "IDIOMATIC" column for the heatmap
if 'IDIOMATIC' in pivot.columns:
    heatmap_data = pivot['IDIOMATIC'].unstack().reindex(columns=register_order)

    plt.figure(figsize=(10, 8))
    sns.heatmap(heatmap_data, annot=True, cmap="Blues", fmt=".0%", cbar_kws={'
    label': '% Idiomatic Usage'})
    plt.title('Heatmap: Intensity of Idiomatic Usage by Word', fontsize=14)
    plt.ylabel('Target Lemma')
    plt.xlabel('Register')
    plt.tight_layout()
    plt.savefig('results_heatmap.png', dpi=300)
    print("Saved 'results_heatmap.png'")
else:
    print("Could not generate heatmap (Label 'IDIOMATIC' missing?)")

print("\nDONE. Open the PNG files to see your thesis results.")
```

E Full AI Documentation

E.1 Gemini 2.5 Flash: SCC Generation Prompts

Date used: 05/02/2026

System Prompt

```
You are a linguistic expert specialising in pragmatics and corpus generation.
Your task is to generate distinct, high-quality sentences for a target word
    based on strict register constraints.
The sentences must be appropriate for a CEFR B1/B2 learner.
```

Batch Prompt

```
Target Word: "{word}"
Batch ID: {batch_num} (Ensure these sentences are unique from generic examples)

Generate a JSON array containing exactly {3 * SENTENCES_PER_REG_PER_BATCH}
    sentences:

1. {SENTENCES_PER_REG_PER_BATCH} sentences: Register HIGH (Formal), Mood
    Question. Context: Workplace/Professional.
2. {SENTENCES_PER_REG_PER_BATCH} sentences: Register LOW (Casual), Mood
    Statement. Context: Friends/Personal/Slang.
3. {SENTENCES_PER_REG_PER_BATCH} sentences: Register NEUTRAL (Direct), Mood
    Imperative. Context: Instructions/Navigation/Rules.

Vary the sentence structure and vocabulary usage.
Output JSON keys: "register", "mood", "sentence".
```

E.2 Gemini 2.5 Flash: Semantic Classification Prompts

Date used: 28/02/2026

System Prompt

```
You are a semantic analyzer. I will provide a list of sentences containing a
    target word.
Classify the usage of the target word in each sentence into exactly one
    category:
1. "LITERAL" (Physical/Core meaning).
2. "IDIOMATIC" (De-lexicalized, Metaphor, Phrasal Verb, or Fixed Phrase).

Return a strict JSON ARRAY of strings.
Example Output: ["LITERAL", "IDIOMATIC", "LITERAL"]
```

Chunk Prompt

```
Target Word: "{word}"
Sentences to classify ({len(chunk_sentences)} items):
{json.dumps(chunk_sentences)}

Return exactly {len(chunk_sentences)} tags in a JSON array.
```

E.3 Gemini 3 Pro: Ideation Prompt

Date used: 20/11/2025

```
I am starting my Bachelor's thesis in English Linguistics. I want to compare
"Textbook English" idiomaticity against sentences generated by an LLM like
Gemini. I am thinking of building two corpora and using a "High/Neutral/Low"
register distinction for the AI side to test whether it can mimic pedagogical
styles. Can you help me brainstorm a formal academic structure for this?
```

Screenshots

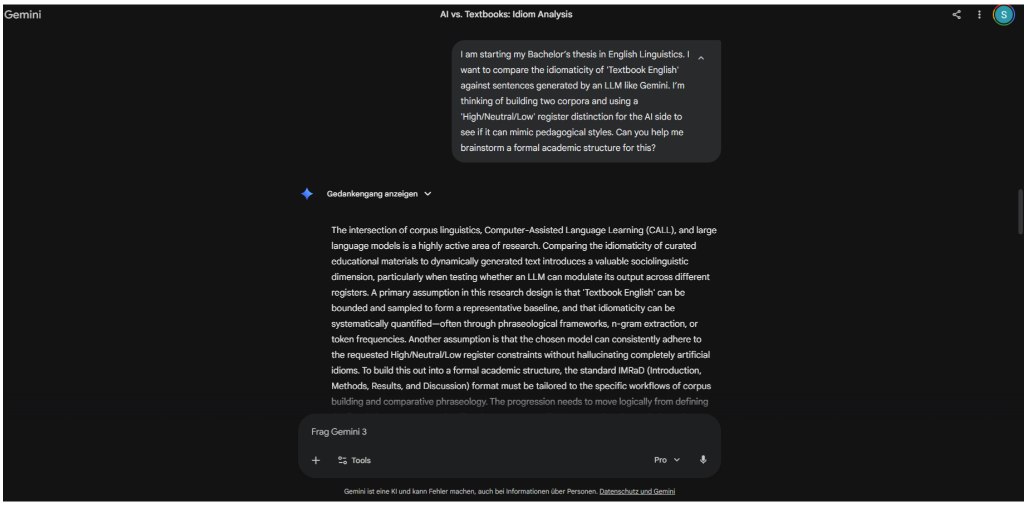


Figure 6: Gemini 3 ideation chat screenshot 1

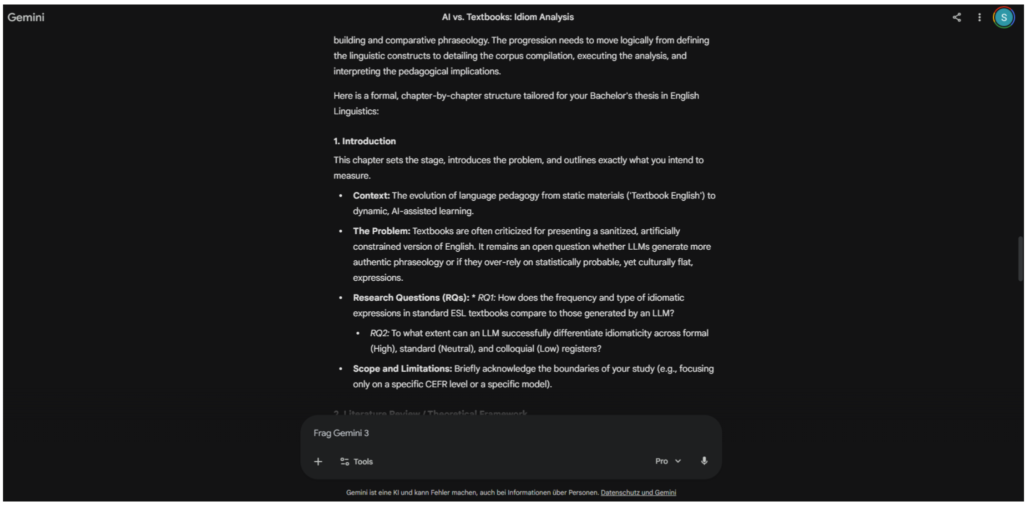


Figure 7: Gemini 3 ideation chat screenshot 2

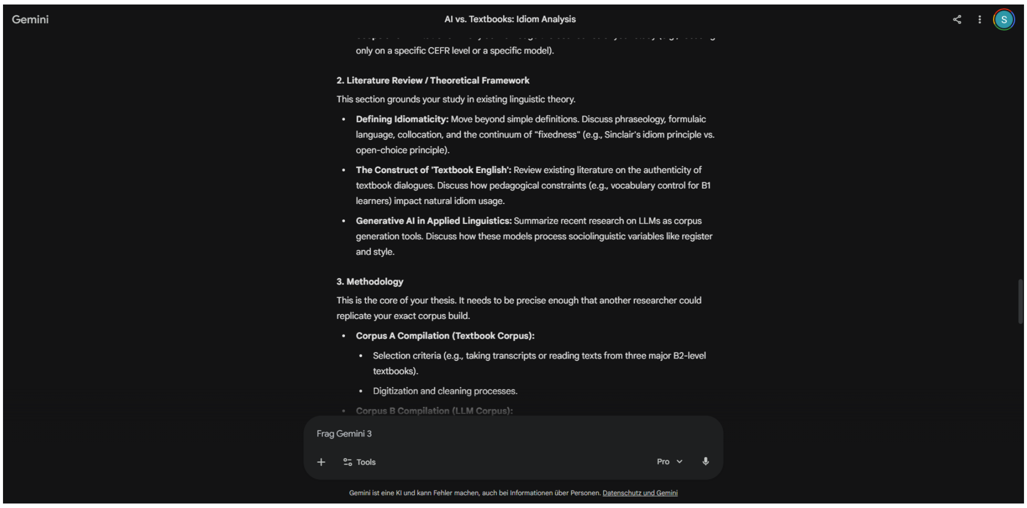


Figure 8: Gemini 3 ideation chat screenshot 3

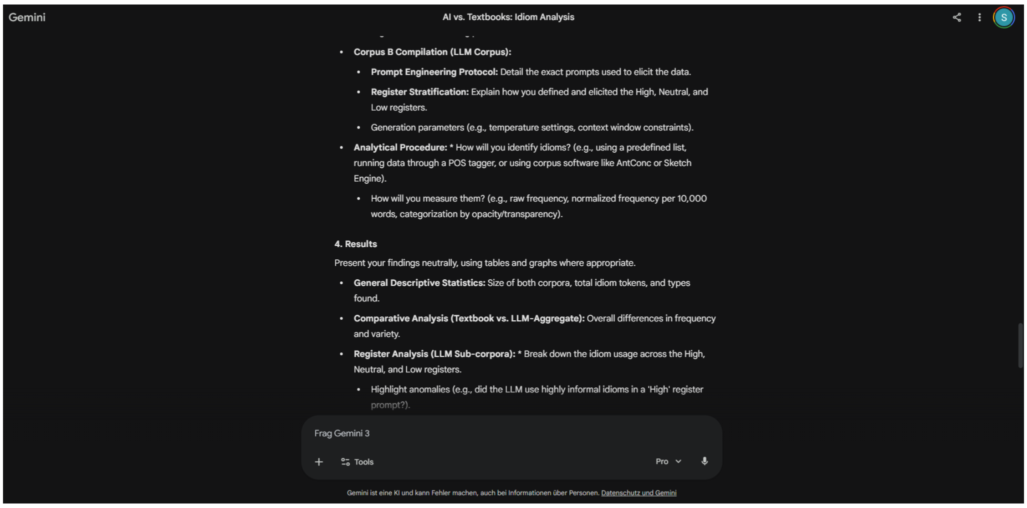


Figure 9: Gemini 3 ideation chat screenshot 4

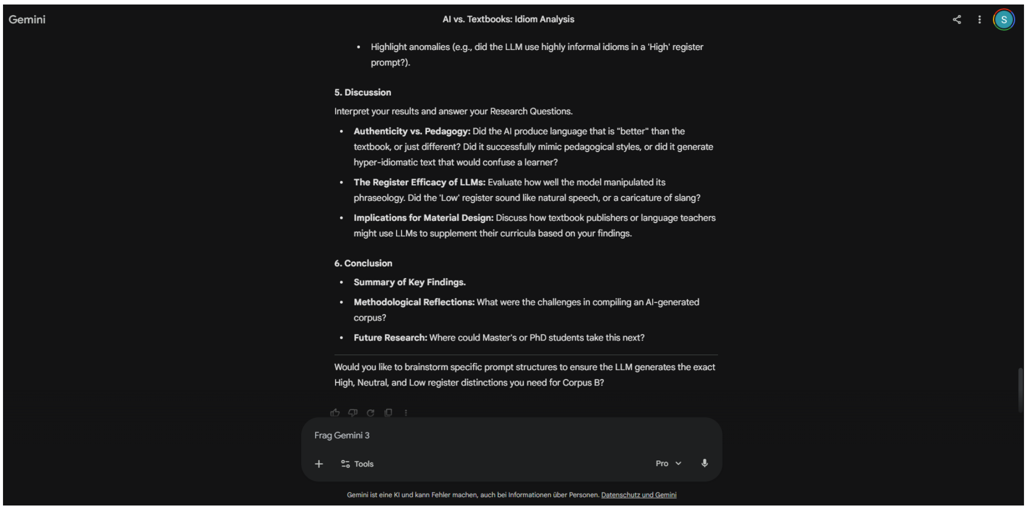


Figure 10: Gemini 3 ideation chat screenshot 5